

TKG-RAG: Knowledge Graph Incorporated with Time Domain for Institutional Setup

by

Hasimunnahar Shanta - 221002585

Jahidul Islam - 221002504

Rifat Hossain Abrar - 221002385

*Capstone Thesis (CSE 400) submitted in partial fulfillment of the
requirements for the degree of*

Bachelor of Science in Computer Science and Engineering

Supervisor

Prof. Dr. Md. Saiful Azad

Co-supervisor

Abdullah Al Masud



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
GREEN UNIVERSITY OF BANGLADESH**

FALL 2025



© Department of *Computer Science and Engineering*,
Green University of Bangladesh
All rights reserved.

DECLARATION

TKG-RAG: Knowledge Graph Incorporated with Time Domain for Institutional Setup

Author *Hasimunnahar Shanta, Jahidul Islam & Rifat Hossain Abrar*

Student ID 221002585, 221002504 & 221002385

Supervisor Prof. Dr. Md. Saiful Azad

Co-supervisor Abdullah Al Masud

We declare that this thesis entitled *TKG-RAG: Knowledge Graph Incorporated with Time Domain for Institutional Setup* is the result of our own work except as cited in the references. The thesis has not been accepted for any degree and is not concurrently submitted in candidature of any other degree.

Hasimunnahar Shanta - 221002585

Jahidul Islam - 221002504

Rifat Hossain Abrar - 221002385

Department of Computer Science and
Engineering
Green University of Bangladesh

Date: January 17, 2026

CERTIFICATE OF ENDORSEMENT

This is to certify that the thesis entitled *TKG-RAG: Knowledge Graph Incorporated with Time Domain for Institutional Setup* by **Hasimunnahar Shanta, Jahidul Islam & Rifat Hossain Abrar** has been successfully defended and examined. Based on the evaluation and discussion held on **January 17, 2026**, we, the undersigned, hereby endorse the thesis for acceptance and approval.

Md. Solaiman Mia
External Member, Board: 3
Assistant Professor and Deputy Director of Student
Affairs
Green University of Bangladesh

Mr. Shoab Alam
Board Member, Board: 3
Lecturer
Department of Department
University

Syed Ahsanul Kabir
Chair, Board: 3
Associate Professor and Chairman
University

Date: January 17, 2026



Department of Computer Science and Engineering
Green University of Bangladesh
Purbachal American City, Kanchan, Rupganj,
Narayanganj-1461, Dhaka, Bangladesh

CERTIFICATE

This is to certify that the thesis entitled **TKG-RAG: Knowledge Graph Incorporated with Time Domain for Institutional Setup**, submitted by **Hasimunnahar Shanta** (Student ID: 221002585), **Jahidul Islam** (Student ID: 221002504) & **Rifat Hossain Abrar** (Student ID: 221002385), an undergraduate student of the **Department of Computer Science and Engineering**, has been examined. Upon recommendation by the examination committee, we hereby accord our approval of it as the presented work and submitted report fulfill the requirements for its acceptance in partial fulfillment for the degree of *Bachelor of Science in Computer Science and Engineering*. *Bachelor of Science in Computer Science and Engineering*.

Prof. Dr. Md. Saiful Azad
Dean, FSE & Director, IQAC
Green University of Bangladesh

Abdullah Al Masud
Research Engineer, FSE
Green University of Bangladesh

Syed Ahsanul Kabir
Associate Professor and Chairman
Department of Computer Science and Engineering
Green University of Bangladesh

Place: Purbachal American City, Kanchan, Rupganj, Narayanganj-1461, Dhaka, Bangladesh

Date: January 17, 2026

ACKNOWLEDGMENTS

First and foremost, we are deeply grateful to **Green University of Bangladesh** to provide opportunities and resources to continue our educational and research efforts. We extend our heartfelt praise for the **Department of Computer Sciences and Engineering** for promoting a learning environment that promotes intellectual development and innovation.

We would like to express our honest thanks to the **Chairman of the Department of Computer Sciences and Engineering** for continuous support and visionary leadership and our educational journey.

Our deepest thanks to all the respected faculty members of the department. His dedication to teaching and mentorship has contributed significantly to the development of our knowledge, skills and professional approaches.

We especially thank our research supervisor and co-supervisor, **Prof. Dr. Md. Saiful Azad Sir** and **Abdullah Al Masud Sir**, for granting us the opportunity to work under their invaluable guidance. Their insight, inspiration, and continuous encouragement have been instrumental at every stage of this research. It has been a great privilege and honor to work under their supervision.

We are always grateful to **our parents** for our unconditional love, prayer, and unbreakable support. Their victims have provided the basis for our education and personal development.

We also want to acknowledge the contribution of our colleagues from our **research team**, whose collaboration, dedication, and teamwork made the journey smooth and more fun.

Special thanks to our friends and well-wishers for their continuous support, courage, and companionship throughout this research journey. Finally, We extend our heartfelt gratitude to everyone who supported us directly or indirectly in successfully completing this research.

Hasimunnahar Shanta, Jahidul Islam & Rifat Hossain Abrar

Department of Computer Science and Engineering

Green University of Bangladesh

Date: January 17, 2026



Dedicated to Almighty Allah —
*All praise to Him for His endless
mercy and guidance.*

*Dedicated to my beloved parents for
their love and support, and to all
who inspired and encouraged me
along the way.*

– Hasimunnahar Shanta

To my ever-loving and
supportive parents,
*whose sacrifices, prayers, and
unconditional love have been
my greatest strength.*

– Jahidul Islam

To my respected teachers,
mentors, and well-wishers,
*whose knowledge and
encouragement have shaped
my academic journey.*

– Rifat Hossain Abrar

ABSTRACT

Large Language Models (LLMs) are widely used for question answering, but their reliance on static knowledge limits factual reliability in time-sensitive settings. Retrieval-Augmented Generation (RAG) mitigates knowledge gaps through external retrieval; however, conventional RAG systems remain weak in handling explicit temporal constraints and time-dependent reasoning. This thesis proposes a temporal knowledge graph-enhanced RAG framework that addresses this limitation by integrating structured temporal facts with LLM-based reasoning agents to improve time-aware question answering. The approach incorporates data ingestion, temporal knowledge graph construction, hybrid retrieval, and controlled LLM-based response generation to support temporal reasoning.

A curated evaluation set of 200 temporal questions was constructed from a researcher profiling dataset, with each question associated with a ground-truth answer and explicit temporal conditions. The system produces a single answer per query and is evaluated using exact-match and overlap-based metrics (Token F1, Char F1, ROUGE-L, Jaccard), list-level F1, and numeric error measures. Experimental results show an Exact Match score of 0.675 and a Token F1 score of 0.7767, with overlap-based metrics consistently exceeding 0.75. Numeric errors remain low (MAE 0.3367, MRE 0.0922), indicating reliable handling of year- and count-based queries. These findings confirm that integrating temporal knowledge graphs and reasoning agents within RAG improves factual reliability and temporal consistency in dynamic question answering scenarios.

Keywords: Temporal knowledge graph; Retrieval-augmented generation; Time-aware question answering; Large language models; LLM-based agents

Contents

1	Introduction	1
1.1	Background of the Study	1
1.1.1	State of the Art in the Modern Context	2
1.1.2	Importance and Relevance of the Topic	3
1.2	Research Problem and Its Context	4
1.3	Motivation	5
1.4	Applications and Practical Use Cases	5
1.5	Research Objectives	7
1.6	Proposed Methodology	7
1.6.1	Data Collection and Temporal Knowledge Graph Construction .	8
1.6.1.1	Dataset Preparation	8
1.6.1.2	TKG Construction	8
1.6.2	Temporal Embedding Learning	8
1.6.2.1	Embedding Model Selection	8
1.6.2.2	Temporal Score Function	9
1.6.3	Time-Aware Retrieval Module	9
1.6.3.1	Query Classification	9
1.6.3.2	Temporal Constraint Extraction	9
1.6.3.3	TKG-Based Retrieval	10
1.6.3.4	Document Retrieval (Optional)	10
1.6.4	RAG-Based Response Generation with Temporal Reasoning . . .	10
1.6.4.1	Context Construction	10
1.6.4.2	Time-Aware Generation	11
1.6.4.3	Post-Filtering (Optional)	11
1.6.5	Evaluation and Benchmarking	11
1.6.5.1	Datasets	11
1.6.5.2	Baseline Comparisons	12
1.6.5.3	Evaluation Metrics	12
1.7	Research Contribution	12
1.8	Financial Planning and Timeline	13

1.8.1	Gantt Chart	14
1.8.2	Gantt Chart Explanation	15
1.8.2.1	Phase 1: Initiation and Foundation (Late 2024 – Q1 2025)	15
1.8.2.2	Phase 2: Implementation Setup (Q2 2025)	16
1.8.2.3	Phase 3: Integration and Completion (Q3 – Q4 2025) . .	16
1.8.2.4	Phase 4: Documentation and Publication (Q3 2025 – Q1 2026)	16
1.8.2.5	Summary	17
1.9	Organization of the Dissertation	17
2	Literature Review	18
2.1	Introduction	18
2.2	Theoretical Background	20
2.2.1	Knowledge Graphs	20
2.2.2	Temporal Knowledge Graphs	21
2.2.3	Retrieval-Augmented Generation Models	21
2.2.4	LLM Reasoning Frameworks and Agent-Based Models	22
2.2.5	Linking the Theories to the Proposed Framework	22
2.3	Related Studies	23
2.3.1	Knowledge Graphs & Temporal Knowledge Graphs (TKGs) . . .	24
2.3.2	Time-Aware RAG & Temporal QA	24
2.3.3	Comparative Summary Table	25
2.3.4	Comparative Analysis of Existing Approaches	26
2.3.5	Summary of Research Gaps	29
3	Proposed Methodology	31
3.1	Introduction	31
3.2	Conceptual Framework	31
3.3	System Architecture	32
3.3.1	Data Ingestion and Preprocessing	34
3.3.2	Embedding Generation and Management	34
3.3.3	Large Language Model Integration and Prompting Strategy . . .	36
3.4	Temporal Knowledge Graph Construction and Updates	37
3.4.1	Node Type Definitions and Properties	37
3.4.2	Edge Type Definitions and Temporal Validity	38
3.4.3	Graph Update and Temporal Validity Management	38
3.5	Hybrid Retrieval Pipeline	39
3.5.1	Search Configuration and Methods	39
3.5.2	Rank Fusion and Re-ranking	40

3.5.3	Temporal and Group Filtering	40
3.5.4	Retrieval Pipeline Execution	41
3.6	Automated Maintenance Routines	41
3.6.1	Entity and Edge Deduplication	41
3.6.2	Entity Summarization	43
3.6.3	Temporal Invalidation	43
3.6.4	Community Detection and Updates	43
3.7	Evaluation Methodology	44
3.7.1	Retrieval Quality Assessment	44
3.7.2	LLM Response Validation	44
3.7.3	Latency and Throughput Monitoring	45
3.7.4	Qualitative User Feedback	45
3.8	Operational Infrastructure and Error Handling	45
3.8.1	Configuration Management	45
3.8.2	Rate Limiting and Exponential Backoff	46
3.8.3	Caching Mechanisms	46
3.8.4	Error Handling and Logging	46
3.9	Limitations and Future Work	47
3.9.1	Current Limitations	47
3.9.2	Planned Extensions	48
3.10	Engineering and Societal Considerations	49
3.10.1	Societal Impacts	49
3.10.2	Environmental and Sustainability Considerations	50
3.10.3	Ethical and Professional Responsibility	50
4	Results and Discussion	52
4.1	Introduction	52
4.2	Evaluation Methodology	52
4.2.1	Dataset and Experimental Setup	52
4.2.2	Evaluation Framework	52
4.2.3	Baseline Systems	53
4.3	Quantitative Results	54
4.3.1	Formal Metric Definitions	54
4.3.2	Overall Performance	54
4.3.3	RAGAS Metric Analysis	55
4.3.4	Answer-Level Metric Analysis	55
4.3.5	Performance Variance Analysis	57
4.4	Comparative Analysis Against Baseline Systems	61
4.4.1	Overall Performance Comparison	61

4.4.2	RAGAS Metrics Comparison	62
4.4.3	Answer-Level Metrics Comparison	63
4.4.4	Multi-Dimensional Performance Analysis	64
4.4.5	Retrieval Quality Analysis	65
4.4.6	Relative Performance Analysis	66
4.4.7	Performance Heatmap Analysis	67
4.4.8	Improvement Magnitude Analysis	68
4.4.9	Statistical Significance and Robustness	69
4.4.10	Implications for Temporal RAG Design	70
4.5	Qualitative Analysis	70
4.5.1	Error Taxonomy	70
4.5.2	Research Objectives Assessment	71
4.5.3	Positioning Within Existing Literature	72
4.5.4	Implications for Temporal Knowledge Systems	72
4.5.5	Limitations and Threats to Validity	72
4.6	Summary	73
5	Conclusion	75
5.1	Summary of the Study	75
5.2	Key Findings and Contributions	76
5.3	Limitations	78
5.4	Future Works	79
5.5	Concluding Remarks	79
	References	82

List of Figures

1.1	Overview of the Framework.	7
1.2	Budget Estimation	13
1.3	Gantt Chart	15
3.1	Time Aware RAG framework with Temporal Knowledge Graphs.	33
4.1	Comparative visualization of RAGAS metrics and answer-level metrics. RAGAS metrics (blue) evaluate retrieval and generation quality, while answer-level metrics (green) assess lexical correctness. Error bars represent standard deviation across 202 questions.	56
4.2	Unified view of evaluation metrics grouped by category. The vertical separator distinguishes RAGAS-based assessment (left) from traditional answer quality metrics (right). The dashed horizontal line indicates the 0.5 threshold for acceptable performance.	56
4.3	Bar chart comparison of mean scores across all eight evaluation metrics. Error bars represent one standard deviation. RAGAS metrics demonstrate stronger consistency (smaller error bars) compared to answer-level metrics, particularly for Context Precision and Answer Correctness. The high variability in BLEU and Exact Match reflects sensitivity to lexical variations in short-form answers.	57
4.4	Distribution analysis of evaluation metrics. Each subplot displays a histogram with kernel density estimation (KDE) overlay and mean indicator. Bimodal distributions in Faithfulness and Exact Match suggest distinct performance regimes—questions either succeed comprehensively or fail substantially. In contrast, Context Precision exhibits strong skew toward high values, indicating consistent retrieval quality. The Answer Relevance distribution shows concentration in the lower range, reflecting the verbosity issues discussed in Section 4.4.	58

4.5	Box plot visualization of metric distributions showing medians (central line), quartiles (box boundaries), and outliers (diamonds). Red diamonds indicate mean values. The compact boxes for Context Precision and Context Recall confirm low variance and high central tendency, while elongated boxes for BLEU Score and Answer Relevancy reveal greater spread. Notably, several metrics exhibit ceiling effects with medians at or near 1.0, indicating frequent perfect scores for specific question categories.	59
4.6	Correlation heatmap revealing relationships between evaluation metrics. Strong positive correlations exist between Context Recall and Faithfulness (0.82), indicating that comprehensive context retrieval directly supports factual grounding. Similarly, F1 Score and Exact Match show high correlation (0.89), reflecting their shared dependence on lexical overlap. Weaker correlations between Answer Relevancy and other metrics (0.3–0.5) confirm that relevancy captures a distinct quality dimension orthogonal to correctness and faithfulness.	60
4.7	Comprehensive metric-by-metric comparison across all three systems. TKG-RAG (blue) demonstrates superior performance on seven of eight metrics, with particular dominance in answer-level metrics. Error bars indicate standard deviation. The proposed system achieves consistent excellence across both retrieval quality and answer generation dimensions.	62
4.8	RAGAS metric comparison across three systems. TKG-RAG demonstrates dominance in retrieval quality metrics (Context Precision, Context Recall) and answer correctness. The lower Answer Relevancy reflects a system design prioritizing temporal accuracy over response conciseness. Error bars represent standard deviation across 202 questions.	63
4.9	Answer-level metric comparison revealing stark performance differences. TKG-RAG achieves near-perfect scores while baseline systems struggle, particularly on Exact Match where both baselines effectively fail. This dramatic disparity underscores the critical importance of temporal modeling for factual correctness in temporal question answering.	64
4.10	Multi-dimensional performance comparison via radar plot. TKG-RAG (blue) exhibits a balanced, expansive profile across all metrics. Graph-based RAG (orange) shows strength in retrieval but weakness in answer generation. Entity-based RAG (purple) demonstrates uniformly limited capability. The visualization confirms TKG-RAG’s comprehensive superiority across evaluation dimensions.	65

4.11	Focused comparison of retrieval quality metrics. TKG-RAG demonstrates superior context precision and recall, indicating effective temporal constraint filtering. The modest gap between TKG-RAG and graph-based RAG suggests structural modeling provides substantial benefit, with temporal specialization offering additional incremental improvement.	66
4.12	Baseline performance expressed as percentage of TKG-RAG scores. The horizontal line at 100% represents TKG-RAG baseline. Both baseline systems demonstrate severe deficiencies in answer-level metrics, with entity-based RAG achieving 0% on Exact Match and graph-based RAG reaching only 17% on F1 Score. RAGAS metrics show stronger relative performance, particularly for graph-based RAG, indicating that structural modeling provides partial benefit without complete temporal awareness.	67
4.13	Performance heatmap providing holistic system-metric visualization. Green indicates strong performance, yellow moderate, and red weak. The distinct horizontal banding reveals systematic performance differences between systems. TKG-RAG demonstrates consistent excellence (green band) while baselines show severe deficiencies (red) in answer generation metrics despite moderate retrieval quality (yellow-green). . .	68
4.14	TKG-RAG improvement percentages over best baseline performance for each metric. Positive bars indicate superiority; negative bars indicate inferiority. The dramatic improvements in answer-level metrics (hundreds to thousands of percent) contrast with modest improvements in RAGAS metrics, highlighting that temporal modeling primarily enhances answer generation accuracy. The negative bar for Answer Relevancy reflects the system’s deliberate verbosity trade-off.	69

List of Tables

1.1	Initial Development Cost (Taka in Thousands)	13
1.2	Total Expenses (Taka in Thousands)	14
1.3	Revenue Model (Taka in Thousands)	14
1.4	Cash Flow Analysis (Taka in Thousands)	14
2.1	Comparative Summary of Key Prior Works in RAG, Temporal Reasoning, and TKG Research	26
2.2	Comparative summary of representative approaches.	26
2.3	Mapping of research gaps to their implications and corresponding thesis objectives.	30
4.1	Comprehensive evaluation metrics on temporal question set (N=202)	54
4.2	System-level performance comparison (N=202 temporal questions)	61
4.3	TKG-RAG improvements over best baseline performance	69

1. Introduction

1.1 Background of the Study

In the era of artificial intelligence (AI), Large Language Models (LLMs) have brought a revolutionary change in the way we work. These models can understand and generate human-like text, making many of our daily tasks easier such as writing, summarizing, translating, coding, and answering questions. Because of their wide capabilities, we have become highly dependent on LLMs. Large Language Models (LLMs) have made significant progress in question answering, consistently demonstrating high accuracy and generalization across open-domain and domain-specific tasks.[1] But despite their power, LLMs have some important limitations. LLMs are trained on a fixed dataset [2], which means they cannot store or access updated facts after their training period. So when new information comes out like recent events, new research findings, or updated statistics LLMs often fail to provide correct or up-to-date answers.

To solve the limitation of LLMs' inability to access updated or external knowledge, Retrieval-Augmented Generation (RAG) was introduced by Lewis et al. in 2020. RAG combined two components retrieval and generation. The model first retrieves relevant information from an external knowledge source and then uses an LLM to generate accurate, fact-based answers.[1] RAG solves this by adding a system that can search for updated and relevant information from outside sources when the model is giving an answer. This helps LLMs provide more correct, reliable, and up-to-date results.[3]

Knowledge Graphs (KGs) are organized data models that show entities and how they are connected, allowing machines to store, understand, and work with complex linked information. It started as academic ideas many years ago and have now become important tools in modern AI. While people were organizing knowledge in a structured way as early as the 1970s, the term "Knowledge Graph" and its wide use became popular more recently, especially after Google introduced its Knowledge Graph in 2012.[4] Real-world facts often change over time, like job positions, events, policies, or political situations. Traditional static Knowledge Graphs cannot represent these changes.

TKGs extend traditional knowledge graphs by associating each fact (triple) with a timestamp or time interval, forming quadruples (subject, relation, object, time). This allows TKGs to represent not just static facts, but also the temporal evolution of entities and their interactions, making them more suitable for real-world, time-sensitive data.[5] Models such as TTransE, DE-SimpleE, and CyGNet are recognized as important milestones in the development of Temporal Knowledge Graphs (TKGs). These models introduce methods to capture how facts and relationships evolve over time, enabling more accurate reasoning and prediction of events as they change. The survey by Ji et al. highlights the significance of incorporating temporal information into knowledge graph representation learning and completion, noting that such temporal models are essential for understanding the dynamic nature of real-world knowledge and for supporting time-sensitive applications. [5] While Retrieval-Augmented Generation (RAG) improves factual accuracy by retrieving external information, it still lacks time-aware retrieval and cannot always detect outdated facts. This limitation creates a research gap.

The aim of our thesis is to address this gap by integrating RAG with Temporal Knowledge Graphs (TKGs), enabling time-sensitive and up-to-date reasoning in generated responses.

1.1.1 State of the Art in the Modern Context

Large Language Models (LLMs), Retrieval-Augmented Generation (RAG), Knowledge Graphs (KGs), and Temporal Knowledge Graphs (TKGs) have become more popular in AI systems in recent years. These tools are now being combined to improve AI's intelligence of data, especially as time and real-world changes are significant.

Models like GPT-4 and other open-source LLMs show strong abilities in understanding language, generating text, and reasoning. But because they are trained on fixed datasets, their knowledge can become outdated, and they may produce incorrect or imagined information especially in fields that change quickly. RAG helps solve this problem by bringing in fresh, relevant information from outside sources. This makes the answers more accurate and flexible, without needing to retrain the LLM. Modern RAG systems use more flexible designs, mix different types of search methods (like vector search and keyword search), and train their retrievers using self-supervised techniques. These improvements help the system find information more accurately and work more efficiently. Vector databases and hybrid search techniques are now basic modern technology. They provide rapid real-time access to large and diverse amounts of data, making the whole structure more flexible and efficient.[3]

Knowledge graphs (KGs) are known as an essential component for visualizing structured, semantic relationships between items, offering complex tasks like question

answering, recommendation systems, and information retrieval in both commercial and educational contexts. Their ability to organize and combine huge and diverse datasets has significantly helped in the development of more intelligent, accessible and easily understood AI systems. Despite these advances, KGs are fundamentally insufficient and require more improvements in information extraction, knowledge representation, and reasoning procedures to meet their goals. Recent research has highlighted the importance of temporal knowledge graphs (TKGs), which use temporal properties to describe the changing nature of facts and relations, offering for more efficient and context-aware reasoning in changing real-world scenarios. Still, major challenges remain, such as effective time-based representation, data shortage reduction, and the requirement for scalable, high-quality graph creation methodologies for a wide range of subsequent applications. Methodological improvement in both KG and TKG research is also essential to establishing knowledge-driven AI and supporting a broader range of developing intelligent systems.[4][5]

1.1.2 Importance and Relevance of the Topic

Retrieval-Augmented Generation (RAG) combined with Temporal Knowledge Graphs (TKGs) is an important improvement to follow in solving the limitations of existing Large Language Models (LLMs), which are limited by static knowledge and tend to generate out-of-date or incorrect responses. Despite their ability to produce language that is both fluent and contextually relevant, traditional LLMs struggle in fields that need current knowledge because they lack tools for accessing and reasoning about frequently changing information. Combining RAG and TKGs allows models to retrieve significant external data while also taking temporal patterns into consideration, which enables exact, time-aware reasoning over changing facts and situations. In research, business, and societal situations, including as decision-making, information retrieval, question answering, and predictive analysis, this method improves the dependability and application of LLMs. This architecture is expected to be very useful for companies and other organizations which require real-time, accurate information, like financial institutions, healthcare facilities, or dynamic information-management applications. The integration also helps AI research by making context-sensitive reasoning more efficient, improving knowledge more accurate, and reducing hallucinations in generated responses.

Recent research in RAG and LLM knowledge modification highlights the benefits of combining retrieval methods with large models to maintain up-to-date knowledge.[1][2] While temporal knowledge graph frameworks offer a comprehensive and visual method for recording the change of entities and relationships over time.[6] Overall, these improvements highlight the research's historical and commercial value while

opening the way to more reliable, flexible, and time-aware artificial intelligence (AI) platforms.

1.2 Research Problem and Its Context

Integrating Retrieval-Augmented Generation (RAG) with Temporal Knowledge Graphs (TKGs) offers a promising solution to a significant challenge in Large Language Models (LLMs): enabling time-sensitive reasoning over evolving knowledge. Although LLMs excel in natural language understanding and text generation, their reliance on static, pre-trained data constrains their ability to provide up-to-date information, often resulting in inaccuracies, hallucinations, and limited temporal reasoning. This limitation is especially critical in rapidly changing domains such as healthcare, finance, and scientific research, where decisions depend on timely and accurate knowledge. [3][7] While RAG approaches improve LLM outputs by retrieving external information, most existing frameworks remain oblivious to the temporal dimension, struggling to differentiate between past and current facts. Traditional Knowledge Graphs (KGs) offer structured relational information but fail to capture the evolution of facts over time, which can lead to ambiguities when reasoning about temporally sensitive events. Temporal Knowledge Graphs (TKGs) overcome this limitation by encoding the dynamics of entities and their relationships, yet their integration with LLMs for real-time, time-aware reasoning is still an open research challenge and technically demanding.[7]

Our goal is to make RAG time-aware using TKG to address four key issues:

- **Event ambiguity:** Disambiguating events that occur at different times but share similar entities or relations.
- **Version confusion:** Distinguishing between multiple versions of facts or entities as they evolve.
- **Update conflicts:** Resolving inconsistencies arising from overlapping or contradictory updates in the knowledge base.
- **Invalid or outdated data:** Preventing the use of obsolete or superseded information in reasoning and generation.

In order to improve the accuracy, dependability, and applicability of LLM outputs in dynamic, real-world scenarios and ultimately enable more reliable and context-sensitive AI systems for people, companies, and society at large it requires that such issues be resolved.

1.3 Motivation

The rapid evolution of information in today's world has created a pressing need for intelligent systems that can provide accurate, time-sensitive, and context-aware answers. Although Large Language Models (LLMs) have transformed the way we interact with AI, their inability to access updated or time-specific knowledge remains a critical limitation. These models are trained on static datasets and therefore often generate outdated, incorrect, or hallucinated responses when new events, policy changes, or evolving facts arise.

Retrieval-Augmented Generation (RAG) introduced a major advancement by enabling LLMs to fetch external documents during inference. However, existing RAG systems do not consider the temporal dimension of knowledge. They treat retrieved information as static, which leads to issues such as retrieving outdated documents or failing to detect time dependent changes. In real-world scenarios such as political events, financial markets, medical guidelines, or organizational changes time plays a fundamental role in determining factual correctness.

Temporal Knowledge Graphs (TKGs) provide a structured way to represent dynamic facts with timestamps, capturing how entities and relationships evolve over time. Integrating TKGs with RAG can enable AI systems to reason not just about what is true, but when it was true. This motivation drives our research: to build a system that enhances factual accuracy, reduces hallucinations, and supports more reliable, time aware decision making.

1.4 Applications and Practical Use Cases

Integrating RAG with Temporal Knowledge Graphs creates a powerful framework with broad practical applicability. Several real-world domains heavily rely on up-to-date, time-dependent information:

1. Government and Policy Updates

- changes in laws, regulations, government positions, and political events.
- Automatically answering citizen queries with the most recent verified information.
- Monitoring changes in public offices, foreign policy decisions, and economic policies.

2. Healthcare and Medical Knowledge Systems

- Ensuring AI systems provide the latest medical guidelines, treatment updates, or vaccine

- protocols.
- Avoiding outdated medical advice that may be harmful.
- Supporting clinical decision-making with validated temporal facts.

3. Finance, Stock Market, and Business Intelligence

- Monitoring financial events, corporate announcements, mergers, or market volatility.
- Producing time-aware insights for stock analysis and business forecasting.
- Reducing risks caused by outdated or incorrect business intelligence.

4. Search Engines and Intelligent Assistants

- Improving factual accuracy in conversational assistants.
- Delivering responses that reflect the newest available information.
- Reducing hallucinations in information-retrieval applications.

5. News and Event Tracking Systems

- Automatically connecting breaking news with historical event timelines.
- Detecting changes in evolving stories such as conflicts, elections, or natural disasters.
- Helping journalists and researchers retrieve chronological knowledge efficiently.

6. Academic Research and Knowledge Management

- Organizing research updates chronologically across domains.
- Supporting students and researchers with time-aware literature exploration.
- Maintaining accurate research timelines and citation tracking.

7. Corporate Knowledge Bases

- Tracking changes in internal company policies, roles, and ongoing projects.
- Providing employees with updated answers about organizational structure.
- Reducing confusion due to outdated internal documentation

1.5 Research Objectives

The primary aim of our research is to enhance the capabilities of Retrieval-Augmented Generation (RAG) by integrating it with Temporal Knowledge Graphs (TKGs) to enable time-aware reasoning in Large Language Models (LLMs). To achieve this, our study focuses on the following three main objectives:

- **Feature Extraction from Knowledge Graphs:** To extract essential features from knowledge graph data, including entities, relationships, and timestamps, which form the foundation for temporal reasoning.
- **Development of a Time-Aware KG-RAG Model:** To design and implement a KG-RAG framework that incorporates temporal reasoning and time-sensitive retrieval, enabling LLMs to generate accurate and up-to-date responses.
- **Performance Evaluation:** To assess the effectiveness of the proposed model by conducting a comprehensive comparison with existing RAG and knowledge graph-based approaches, evaluating metrics such as accuracy, temporal consistency, and reasoning capability.

1.6 Proposed Methodology

Our methodology introduces a hybrid framework that integrates Retrieval-Augmented Generation (RAG) with Temporal Knowledge Graphs (TKGs) to enable time-aware, factual, and context-relevant question answering. This section outlines each stage of the system development process.

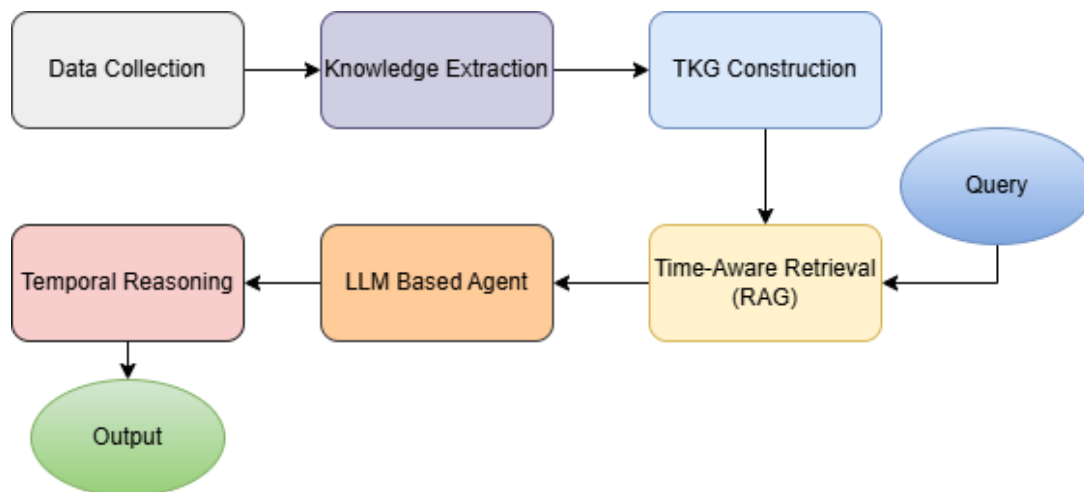


Figure 1.1: Overview of the Framework.

1.6.1 Data Collection and Temporal Knowledge Graph Construction

1.6.1.1 Dataset Preparation

To build a temporal reasoning system, datasets must include:

- **Entities:** people, places, organizations
- **Relations:** `works_at`, `married_to`, `elected_as`, etc.
- **Temporal information:** timestamps or time intervals

Sources of data may include:

- Public datasets: ICEWS, GDELT, Wikidata5M, YAGO-TI
- Domain-specific datasets: political timelines, medical guideline updates, corporate events, financial reports, etc.

1.6.1.2 TKG Construction

Each fact is represented as a temporal quadruple:

(subject, relation, object, time)

Example:

(Barack Obama, `elected_as`, US President, 2008)

The TKG is further processed to:

- Normalize timestamps (day, month, year)
- Resolve entity ambiguities
- Convert structured data into graph format

This structured representation enables effective temporal reasoning and event prediction.

1.6.2 Temporal Embedding Learning

1.6.2.1 Embedding Model Selection

A Temporal Knowledge Graph Embedding (TKGE) model is trained to encode entities, relations, and their temporal dynamics. Popular embedding models include:

- TTransE
- DE-Simple

- TimE
- CyGNet
- TELEGRAPH
- TLT-KGE (focus model in this study)

These models jointly learn:

- Entity embeddings
- Relation embeddings
- Time embeddings

1.6.2.2 Temporal Score Function

The embedding model produces a scoring function that evaluates the validity of a fact at a specific time:

$$\text{score}(e_s, r, e_o, t) \rightarrow \text{higher value means more likely true at time } t$$

This enables the system to:

- Retrieve relevant time-specific facts
- Filter outdated or incorrect information
- Predict missing or future events

1.6.3 Time-Aware Retrieval Module

This module introduces temporal reasoning into the RAG pipeline.

1.6.3.1 Query Classification

Incoming user queries are classified into:

- **Temporal queries:** e.g., “Who was the president of India in 2018?”
- **Static queries:** e.g., “What is photosynthesis?”

1.6.3.2 Temporal Constraint Extraction

For temporal queries, the system extracts:

- **Explicit time expressions:** “in 2020”, “during COVID-19”
- **Implicit time expressions:** “current president”, “recent updates”

1.6.3.3 TKG-Based Retrieval

The temporal embedding model retrieves the most relevant time-aware facts. The procedure includes:

1. Identifying entities in the query
2. Retrieving temporally relevant relations
3. Ranking results using the temporal score function
4. Selecting the top- k temporally valid facts

Example (for query: “*Who is the CEO of Twitter now?*”):

(Elon Musk, *CEO_of, Twitter*, 2022–2023)

(Linda Yaccarino, *CEO_of, Twitter*, 2023–Present)

TKG retrieval identifies the current CEO based on the timestamp.

1.6.3.4 Document Retrieval (Optional)

To complement TKG retrieval, the system may perform:

- Vector similarity search on PDFs, news, or webpages

This provides additional factual grounding for complex queries.

1.6.4 RAG-Based Response Generation with Temporal Reasoning

1.6.4.1 Context Construction

The system constructs the model input using:

- Time-relevant TKG facts
- Supporting retrieved documents
- Event timelines (if required)

1.6.4.2 Time-Aware Generation

The LLM generates the final answer by:

- Using temporal evidence as grounding
- Avoiding outdated or incorrect facts
- Maintaining consistency with the correct time period
- Reasoning about event transitions or updates

This significantly reduces:

- Hallucination
- Temporal inconsistencies
- Incorrect summaries

1.6.4.3 Post-Filtering (Optional)

A temporal validation step checks:

- Whether the answer matches temporal evidence
- Whether the time period is correct
- Whether outdated facts were included

If mismatches occur, the answer is automatically corrected.

1.6.5 Evaluation and Benchmarking

1.6.5.1 Datasets

Evaluation is performed using datasets involving:

- Political changes
- Corporate roles
- Historical updates
- Real-world time-sensitive events

1.6.5.2 Baseline Comparisons

The proposed system is compared against:

- Standard RAG
- Static KG-based RAG
- Vanilla language models (GPT, LLaMA, Mistral)
- Pure TKG models (DE-Simple, TTransE)

1.6.5.3 Evaluation Metrics

Key evaluation metrics:

- **Temporal Accuracy:** correctness of time-aware answers
- **Factual Consistency:** alignment with ground truth
- **Precision and Recall:** retrieval effectiveness
- **Hallucination rate:** incorrect or fabricated content
- **Answer relevance:** semantic and contextual validity
- **User satisfaction score** (optional)

1.7 Research Contribution

This research presents a novel framework that combines Retrieval Augmented Generation (RAG) with Temporal Knowledge Graphs (TKGs) to enable dynamic, time-aware reasoning in Large Language Models (LLMs). The key contributions are:

1. **Agent-Guided, Time-Aware KG-RAG Model:** We introduce an intelligent agent to guide reasoning over temporal knowledge graphs, allowing LLMs to generate accurate, up-to-date, and context-sensitive responses.
2. **Addressing Temporal Challenges:** The framework resolves critical issues such as event ambiguity, version confusion, update conflicts, and invalid or outdated data, which limit existing RAG and KG approaches.
3. **Efficient Feature Extraction:** Methods are developed to extract entities, relationships, and timestamps from knowledge graphs, supporting dynamic, agent-driven reasoning.

4. **Improved Performance:** The proposed model is evaluated against existing approaches, demonstrating enhanced accuracy, temporal consistency, and reliability.
5. **Real-World Applicability:** By enabling agent-based, time-sensitive reasoning, this framework supports trustworthy AI systems for applications in healthcare, finance, research, and other domains requiring dynamic knowledge management.

Real-World Applicability: By enabling agent-based, time-sensitive reasoning, this framework supports trustworthy AI systems for applications in healthcare, finance, research, and other domains requiring dynamic knowledge management.

1.8 Financial Planning and Timeline

Table : Initial development Cost (taka-In Thousands)				
	Year -01			
	Q1	Q2	Q3	Q4
Microsoft Azure	0	0	0	0
ChatGPT API (10,000 Token)	2.7	2.7	2.7	2.7
Domain name cost	0.25	0.25	0.25	0.25
Total cost	2.95	2.95	2.95	2.95

Table : Total Expenses (taka-In ThousandS)				
	Year -01			
	Q1	Q2	Q3	Q4
Initial Development Cost	2.95	2.95	2.95	2.95
Utility Cost	1	1	1	1
Total Expenses	3.95	3.95	3.95	3.95

Table : Revenue Model (taka-In Thousands)				
	Year -01			
	Q1	Q2	Q3	Q4
User Subscription (per user/month)Xnum of users	(0.25*3)*5=3.75	3.75	3.75	3.75
Subscription Cost	2*5=10	10	10	10
Total Cost	13.75	13.75	13.75	13.75

Table : Cash flow analysis (taka-In Thousands)				
	Year -01			
	Q1	Q2	Q3	Q4
Total Revenue	13.75	13.75	13.75	13.75
Total Expenses	3.95	3.95	3.95	3.95
Cash flow	9.8	9.8	9.8	9.8
Cumulative Cash flow	9.8	19.6	29.4	39.2

Figure 1.2: Budget Estimation

This section presents the financial planning for Year 01, including the initial development cost, total operational expenses, estimated revenue, and cash flow analysis. All financial values are expressed in thousands of Bangladeshi Taka.

Table 1.1: Initial Development Cost (Taka in Thousands)

Cost Items	Q1	Q2	Q3	Q4
Microsoft Azure	0	0	0	0
ChatGPT API (100,000 Tokens)	2.70	2.70	2.70	2.70
Domain Name Cost	0.25	0.25	0.25	0.25
Total Cost	2.95	2.95	2.95	2.95

Table 1.2: Total Expenses (Taka in Thousands)

Expense Items	Q1	Q2	Q3	Q4
Initial Development Cost	2.95	2.95	2.95	2.95
Utility Cost	1.00	1.00	1.00	1.00
Total Expenses	3.95	3.95	3.95	3.95

Table 1.3: Revenue Model (Taka in Thousands)

Revenue Items	Q1	Q2	Q3	Q4
User Subscription Revenue	3.75	3.75	3.75	3.75
Subscription Cost	10.00	10.00	10.00	10.00
Total Revenue	13.75	13.75	13.75	13.75

Table 1.4: Cash Flow Analysis (Taka in Thousands)

Cash Flow Metrics	Q1	Q2	Q3	Q4
Total Revenue	13.75	13.75	13.75	13.75
Total Expenses	3.95	3.95	3.95	3.95
Cash Flow	9.80	9.80	9.80	9.80
Cumulative Cash Flow	9.80	19.60	29.40	39.20

Summary

The initial development cost remains constant at 2.95 k Taka per quarter. When combined with the utility cost, the total operating expense stands at 3.95 k Taka per quarter. The projected revenue of 13.75 k Taka per quarter ensures a positive cash flow of 9.8 k Taka, resulting in a cumulative annual cash flow of 39.2 k Taka. Overall, the project demonstrates strong financial feasibility from the first quarter.

1.8.1 Gantt Chart

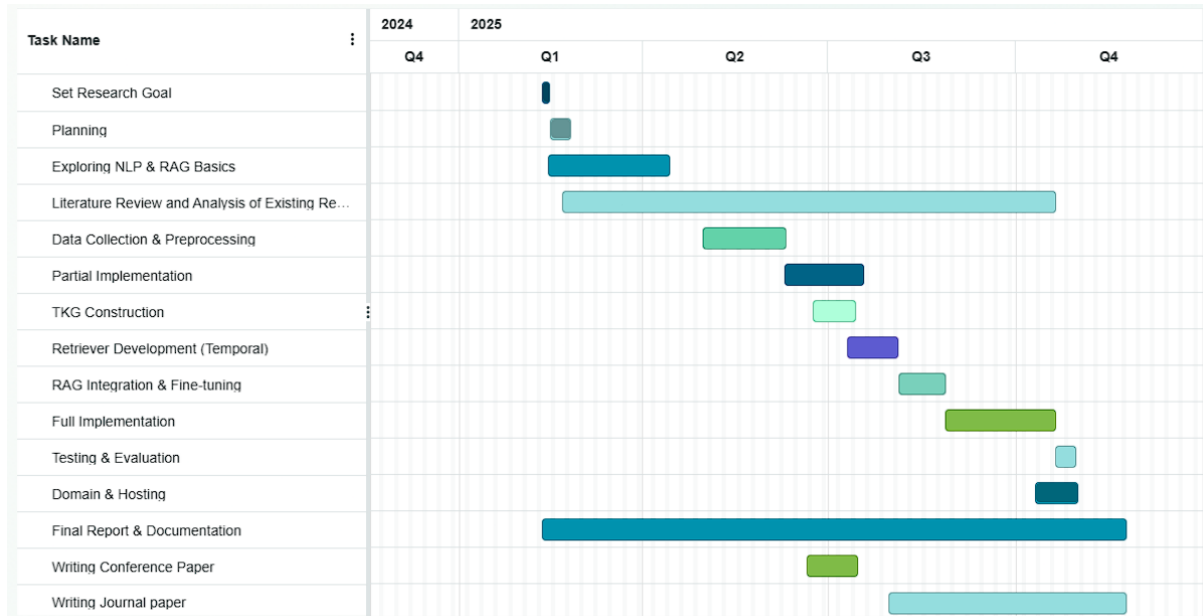


Figure 1.3: Gantt Chart

1.8.2 Gantt Chart Explanation

The Gantt chart presented in this study illustrates the complete project timeline extending from Q4 2024 to Q4 2025. The workflow is organized into four major phases: (i) Initiation and Foundation, (ii) Implementation Setup, (iii) Integration and Completion, and (iv) Documentation and Publication. Each phase is aligned with the research objectives of developing an optimized query classification and Retrieval-Augmented Generation (RAG) system supported by a Temporal Knowledge Graph (TKG).

The Gantt chart presented in this study illustrates the complete project timeline extending from Q4 2024 to Q4 2025. The workflow is organized into four major phases: (i) Initiation and Foundation, (ii) Implementation Setup, (iii) Integration and Completion, and (iv) Documentation and Publication. Each phase is aligned with the research objectives of developing an optimized query classification and Retrieval-Augmented Generation (RAG) system supported by a Temporal Knowledge Graph (TKG).

1.8.2.1 Phase 1: Initiation and Foundation (Late 2024 – Q1 2025)

Set Research Goal: Late Q4 2024 to early Q1 2025. This stage involves defining the primary research questions and objectives.

Project Planning: Conducted in early Q1 2025 to structure the workflow and allocate the required resources.

Exploring NLP and RAG Basics: Early to late Q1 2025. This step develops foundational understanding of NLP methods, RAG architectures, and relevant machine

learning techniques.

Literature Review and Existing Research Analysis: Extending from late Q1 2025 to Q3 2025, this is one of the most prolonged tasks. It includes examining prior studies on retrieval models, query classification, temporal reasoning, and knowledge graph embedding techniques.

1.8.2.2 Phase 2: Implementation Setup (Q2 2025)

Data Collection and Preprocessing: Mid to late Q2 2025. This task includes dataset acquisition, cleaning, formatting, and preparing the data for model training.

Partial Implementation: Early to mid Q3 2025. Initial modules of the system are implemented to validate design direction.

TKG Construction: Mid to late Q3 2025. The Temporal Knowledge Graph is constructed to support temporal reasoning in the retrieval process.

Temporal Retriever Development: Late Q3 2025 to early Q4 2025. This component focuses on developing the retrieval mechanism with temporal awareness and optimized query classification.

1.8.2.3 Phase 3: Integration and Completion (Q3 – Q4 2025)

RAG Integration and Fine-Tuning: Late Q3 to early Q4 2025. The retrieval module is integrated with the generation model, followed by parameter tuning to improve system accuracy.

Full System Implementation: Mid to late Q4 2025. All individual modules are merged into a unified production-ready system.

Testing and Evaluation: Late Q4 2025 to end of Q4 2025. The system is evaluated using benchmark datasets and metrics to validate performance.

Domain and Hosting Setup: Completed in very late Q4 2025. The final deployment and accessibility configuration are performed.

1.8.2.4 Phase 4: Documentation and Publication (Q3 2025 – Q1 2026)

Conference Paper Writing: Early to mid Q4 2025. A concise manuscript is prepared for conference submission.

Journal Paper Writing: Late Q4 2025 to Q1 2026. A detailed research article intended for journal publication.

Final Report and Documentation: Beginning in early Q1 2025 and continuing through Q4 2025, this ongoing task includes writing the thesis chapters, documenting methodologies, recording experiments, and preparing the final thesis report.

1.8.2.5 Summary

The timeline demonstrates that several tasks, such as literature review and documentation, run parallel throughout the year. Significant time is allocated to TKG construction and RAG integration, reflecting their importance in achieving the research objectives. The project reaches its functional completion in Q4 2025 with full system implementation, testing, and deployment.

1.9 Organization of the Dissertation

Chapter 1 presents the introduction, background, and motivation of the research. It outlines the research problem, objectives, scope, and the significance of integrating temporal reasoning into RAG systems.

Chapter 2 provides a comprehensive literature review, covering existing work on Natural Language Processing (NLP), query classification, knowledge graphs, temporal reasoning, and RAG architectures. It highlights current limitations and identifies research gaps that motivate the proposed approach.

Chapter 3 details the methodology, including dataset preparation, construction of the Temporal Knowledge Graph, temporal embedding learning, query classification, temporal retrieval, and integration with RAG. Each component is explained with relevant mathematical formulations and architectural diagrams.

Chapter 4 discusses the implementation and experimental setup. It includes system design, model configurations, hyperparameters, tools and frameworks used, and the workflow for integrating temporal retrieval into the RAG pipeline.

Chapter 5 presents the results and evaluation. Experimental findings are analyzed using quantitative metrics such as temporal accuracy, retrieval precision, hallucination rate, and comparative performance against baseline models.

Chapter 6 concludes the dissertation by summarizing the research contributions, discussing limitations, and suggesting potential directions for future work, including extensions to multimodal temporal KGs and advanced dynamic retrieval models.

This structured organization ensures a logical flow from foundational concepts to implementation, evaluation, and final conclusions, enabling readers to clearly follow the development and validation of the proposed temporal-aware RAG framework.

2. Literature Review

2.1 Introduction

In recent years, artificial intelligence systems have increasingly been deployed in real-world environments where knowledge evolves over time and decisions must be made under temporal constraints. Traditional data-driven models, however, often assume static knowledge representations, limiting their ability to reason accurately in dynamic and time-sensitive contexts[8]. Addressing this challenge has led to rapid advancements in five interrelated research areas: Large Language Models (LLMs), Retrieval-Augmented Generation (RAG), Knowledge Graphs (KGs), Temporal Knowledge Graphs (TKGs), and agent-based reasoning frameworks. Together, these paradigms form the foundation of modern time-aware reasoning systems.

Over the past decade, LLMs have demonstrated remarkable capabilities in natural language understanding and generation, achieving state-of-the-art performance across a wide range of tasks. Despite these advances, prior studies highlight fundamental limitations. Lewis et al. note that LLMs are trained on static corpora, making them prone to hallucinations, outdated responses, and poor handling of evolving factual knowledge[9]. To mitigate these issues, Retrieval-Augmented Generation frameworks have been introduced,[9] enabling models to ground their outputs in external knowledge sources. Recent surveys and studies, including those by Gao et al., show that RAG significantly improves factual accuracy and interpretability[10]. Nevertheless, most existing RAG systems rely on fixed document retrieval, which fails to capture the temporal dynamics inherent in many real-world knowledge domains[10].

Parallel to RAG research, Knowledge Graphs have emerged as a structured and interpretable means of representing complex relational information[8]. Building on this, Temporal Knowledge Graphs explicitly encode time-dependent relationships[11], allowing models to reason over events, state changes, and evolving entities. Research in temporal reasoning such as the work of Cai et al. (IJCAI 2022) and Qian et al. (EMNLP 2024) demonstrates the importance of incorporating temporal signals into question answering and reasoning tasks[11, 12]. However, the integration of TKGs into LLM-based RAG frameworks remains limited, and existing solutions often lack scalability or real-time adaptability[10].

More recently, agent-based reasoning approaches have gained attention for their ability to decompose complex tasks, coordinate multiple reasoning steps, and dynamically interact with external tools[13]. When combined with LLMs, agent frameworks offer a promising pathway toward adaptive, time-aware retrieval and reasoning systems[14]. Despite this potential, current research has not fully explored how agent-driven mechanisms can jointly leverage temporal knowledge graphs and retrieval-augmented architectures to support rapid, accurate, and temporally consistent decision-making[13].

This literature review aims to critically examine how existing approaches address knowledge-intensive and time-sensitive reasoning, and where they fall short. Specifically, it investigates the limitations of static retrieval mechanisms, the underutilization of temporal knowledge representations, and the lack of cohesive agent-based integration within RAG frameworks. The central research challenge addressed in this thesis is the development of a retrieval and reasoning model that is temporally responsive, dynamically adaptable, and suitable for real-world applications requiring precise and timely decisions.

The studies reviewed in this chapter were identified through a structured search strategy across major academic databases, including Scopus, IEEE Xplore, Web of Science, ACL Anthology, and Google Scholar, ensuring a high standard of academic rigor. Keywords such as “Retrieval-Augmented Generation,” “Temporal Knowledge Graph,” “time-aware LLM,” “agent-based reasoning,” “dynamic retrieval systems,” and “temporal question answering” guided the selection process. Only peer-reviewed English-language publications with clear methodological contributions to LLMs, RAG models, KGs/TKGs, or agent-based reasoning were included. Foundational surveys, such as Ji et al.’s comprehensive review[8] of Knowledge Graph research (IEEE TNNLS 2022), were also incorporated to provide historical and conceptual context.

The rest of the chapter continues in an organized way, beginning with the basic concepts and finishing with the recognition of research gaps. The scientific basis of LLMs, RAG frameworks, KGs, TKGs, and agent-based reasoning are explained in Section 2.2. Theoretical and application research are discussed in Section 2.3, organized based on common themes and technological similarities. Section 2.4 compares existing approaches, evaluating their advantages and limitations. Finally, Section 2.5 brings these insights together to highlight the major gaps that justify the direction of this thesis. Overall, these sections establish the need for a time-sensitive KG-RAG framework enhanced with agent reasoning to effectively manage the dynamic nature of evolving information landscapes.

2.2 Theoretical Background

This section discusses the theoretical foundations that support the construction of a temporal knowledge graph–based retrieval-augmented generation structure. Recent improvements in natural language processing, knowledge representation, and time-aware reasoning has established various conceptual and computational models that directly influence the structure of this study. To establish an integrated framework for the proposed system, this chapter combines four essential conceptual foundations: relational knowledge graphs, temporal knowledge graphs, retrieval-augmented generation models, and agent-driven reasoning frameworks for large language models. When combined, these theories provide a framework for the storage, retrieval, analysis, and embedding of factual data in contexts that change over time.

The purpose of this part is not just to summarize existing ideas but also to evaluate their hypotheses, advantages and limitations. Several shortcomings in method are shown by a comprehensive examination of these models, including their lack of temporal dependence in retrieval and reasoning processes. These ideas create the conceptual foundation to the methodological decisions outlined in Chapter 3.

2.2.1 Knowledge Graphs

Knowledge Graphs (KGs) provide a principled and structured mechanism for representing factual knowledge through interconnected entities and relations[8]. Typically expressed as triples (h, r, t) [15], KGs encode semantic dependencies in a machine-interpretable format, enabling efficient graph traversal, link prediction, and diverse reasoning tasks[16]. Owing to their interpretability and logical consistency, KGs underpin a wide range of applications, including information retrieval, recommendation systems, and knowledge-intensive question answering[17].

Formally, a KG is defined as a directed graph $G = (V, E)$, where V denotes the set of entity nodes and E represents labeled edges specifying semantic relations[17]. Each edge captures a relational instance between two entities. A substantial body of embedding approaches including translational, semantic-matching, and geometric models projects entities and relations into continuous vector spaces to facilitate scalable computational reasoning. Compared with purely text-based representations, the explicit structure of KGs enhances interpretability and mitigates hallucination[8].

Despite these advantages, classical KGs exhibit two significant limitations. First, they rely on a static view of the world and thus fail to capture the temporal evolution of facts [18]. Second, many real-world domains—such as political processes, financial markets, epidemiological patterns, and policy transitions—are fundamentally time-dependent [19]. As a result, static KGs are often insufficient for applications requiring temporal or dynamic reasoning[18, 19].

In this study, static KGs serve as the conceptual foundation for developing temporal knowledge representations. Their structured form motivates the design of a more expressive temporal model that supports retrieval-augmented generation in evolving knowledge environments.

2.2.2 Temporal Knowledge Graphs

Temporal Knowledge Graphs (TKGs) extend conventional KGs by enriching each fact with temporal information such as timestamps, intervals, or event durations[20]. By representing a fact as a quadruple (h, r, t, τ) , TKGs capture how relations arise, evolve, and terminate across time[21]. This temporal structure enables queries such as “What was true at a specific moment?” or “How did a relation change over different periods?”, which static KGs are unable to support[20, 21].

Formally, a TKG is defined as a time-indexed graph $G_\tau = (V, E_\tau)$, where the edge set E_τ varies with time and encodes the temporal validity of relational facts. Depending on the modelling assumptions, time can be represented either as discrete steps or as continuous intervals. Event-centric TKG formulations further incorporate contextual attributes such as location, event type, or causal relations, enabling more expressive temporal reasoning and mitigating ambiguity caused by outdated or context-free facts.

Despite their advantages, TKGs introduce several challenges. Temporal sparsity results in many time-stamped relations appearing infrequently, making it difficult for learning algorithms to generalize. Moreover, temporal inference such as predicting historical changes or forecasting future events requires complex sequence modeling. While prior work provides strong foundations, the integration of temporal graph reasoning with natural language understanding remains an open challenge.

In this study, TKGs are crucial because they allow the retrieval module of the RAG framework to provide the language model with time-specific evidence. This temporal grounding reduces hallucinations and improves reasoning in scenarios where historical evolution or temporal ordering is essential. Consequently, TKGs constitute a core component of the proposed system.

2.2.3 Retrieval-Augmented Generation Models

Retrieval-Augmented Generation (RAG) models integrate external knowledge retrieval with generative language models to enhance factual accuracy and reduce hallucinations[9, 22]. In a typical RAG pipeline, a retriever first identifies relevant documents, passages, or graph nodes, and a generator subsequently uses this evidence to produce grounded and context-aware responses[9]. This architecture mitigates the limitations of purely parametric language models, which often struggle to store and recall broad and rapidly evolving information[23].

Structurally, RAG consists of two modules: (i) a *retriever*, commonly implemented using dense vector similarity, hybrid indexing, or graph-based traversal; and (ii) a *generator*, typically a large language model that conditions its outputs on retrieved evidence[22]. While RAG improves factual grounding, its performance remains constrained by the quality and temporal relevance of retrieved content. Traditional RAG systems are largely time-agnostic and do not distinguish between outdated and up-to-date information, which is problematic in domains where temporal accuracy is essential, such as financial reporting, policy analysis, scientific updates, and news understanding[9].

Integrating Temporal Knowledge Graphs into the retrieval stage addresses this limitation. Time-aware retrieval ensures that the generator receives evidence that is both semantically relevant and temporally valid, leading to more accurate responses for time-sensitive queries[20, 21].

2.2.4 LLM Reasoning Frameworks and Agent-Based Models

Recent advances in language model reasoning have sparked the development of agent-based frameworks in which models operate through iterative cycles of observation, reasoning, action execution, and self-reflection[24, 25]. These agents can interact with external tools including databases, knowledge graphs, search engines, and APIs to perform complex reasoning tasks beyond text-only processing[25]. Such frameworks provide greater interpretability by exposing intermediate reasoning steps[24].

A typical agent system comprises: (i) a reasoning module that selects appropriate actions, (ii) an execution module that interfaces with external tools or environments, (iii) a memory or context module that stores accumulated evidence, and (iv) a reflection mechanism that evaluates intermediate outputs before producing the final response[25, 24]. Despite their expressive power, agent systems face challenges such as hallucinated tool calls, redundant reasoning paths, and difficulty handling long inferential chains[25]. Moreover, most agents lack inherent temporal logic, limiting their ability to reason about dynamic or evolving information unless supported by an external temporal knowledge source[20].

In this study, agent-based reasoning provides a structured approach for navigating the Temporal Knowledge Graph, selecting temporally appropriate nodes, and incorporating them into the generation pipeline. This integration enhances interpretability and ensures temporal alignment between retrieval and response generation.

2.2.5 Linking the Theories to the Proposed Framework

Together, these theoretical foundations underpin the framework developed in this thesis. Knowledge graphs offer structural clarity; temporal knowledge graphs provide

chronological grounding; RAG contributes a mechanism for integrating external evidence; and agent-based reasoning enables controlled interaction with the temporal graph. The limitations identified particularly the lack of temporal awareness in conventional retrieval directly motivate the design decisions presented in Chapter 3, where these insights are operationalized into a comprehensive temporal RAG architecture.

2.3 Related Studies

Retrieval-Augmented Generation (RAG) is a core approach in knowledge-intensive NLP. It combines large language models with external retrieval to improve factual accuracy, contextual relevance, and resistance to hallucinations. Lewis et al. (2020) proposed the RAG framework, integrating neural retrievers with sequence-to-sequence generators. Their technique supports results in current, non-parametric memory, delivering large advantages in open-domain question answering.[3] Subsequent surveys have carefully evaluated the growth of RAG, showing its quick adoption across varied applications and its role in reducing the difficulties of static, parametric LLMs. RAG systems are often analyzed on benchmarks such as Natural Questions (NQ), TriviaQA, and KILT, which provide large-scale, knowledge-intensive tasks for testing retrieval and generation quality.[26] Evaluation factors consist of both retrieval and generation components, including relevance, accuracy, faithfulness, and fluency. Recent studies highlight the necessity for standardized evaluation methodologies, as the hybrid character of RAG delays the evaluation of system performance. Specialized metrics like BERTScore and RAGAS assess contextual relevance and factual consistency. Reference-free frameworks such as RAGAS and ARES help address the limited availability of high-quality ground truth labels.[27]

RAG concepts use a wide range of retrieval algorithms, including sparse and dense retrievers, hybrid approaches, and graph-based retrieval for structured knowledge sources. Dense retrievers, such as those based on dual-encoder models, are growing due to their scalability and semantic matching capabilities. To increase precision and handle complex queries, advanced retrieval approaches use reranking, multi-hop retrieval, and agentic planning. However, most RAG systems are still limited by static retrieval pipelines, which are frequently temporally blind and incapable of adapting to developing knowledge bases.[26]

Despite significant improvements, RAG systems face constant challenges. Static retrieval mechanisms and temporal lack of awareness limit the ability to give time-sensitive or contextually updated information, which leads to an ongoing existence of hallucinations even when external knowledge is available. Methodological assumptions, such as the independence of retrieval and generating modules, might hide error propagation and hamper consistency. Furthermore, reproducibility is made harder by

the lack of standardized benchmarks and the expensive expense of creating accurate datasets. Current research also identifies shortcomings in the integration of temporal reasoning, dynamic knowledge improvements, and efficient retrieval of messy or irrelevant data.[28]

2.3.1 Knowledge Graphs & Temporal Knowledge Graphs (TKGs)

Knowledge graphs (KGs) serve as a core representation for structured, relational knowledge, underpinning tasks from search to question answering [4, 5]. Early surveys document wide-ranging KG uses semantic search, recommendation, and intelligent QA and stress the need for precise representations and scalable construction pipelines [4, 5]. Ji et al. offer a detailed taxonomy of KG representation learning, spanning embeddings, path-based inference, and rule reasoning, and note rising interest in temporal variants to reflect evolving facts [5]. Temporal Knowledge Graphs (TKGs) attach timestamps to facts, allowing models to track changing relationships. Cai et al. review TKG completion methods, grouping them into static and dynamic families and spotlighting advances such as TNTComplEx, DE-Simple, and ChronoR for temporal link prediction [6]. These approaches draw on tensor factorization, temporal embedding schemes, and recurrent designs to encode time-dependent signals. Datasets like ICEWS, GDELT, and Wikidata5M Temporal are common benchmarks, each differing in temporal resolution and event coverage. Persistent challenges remain. Many models falter on fine-grained or irregular time intervals, exposing limits in temporal granularity. Leakage—future information influencing past predictions—undermines evaluation credibility. Most TKG methods stay detached from LLMs, missing chances for richer semantics and flexible reasoning. Assumptions of temporal stationarity and constrained capacity risk misfitting, while added complexity can hinder scalability and interpretability. Evaluation practices are still uneven, making cross method comparisons difficult.

2.3.2 Time-Aware RAG & Temporal QA

TimeR⁴, a novel framework that combines time-aware retrieval with LLMs for temporal knowledge graph question answering (TKGQA), is presented by Qian et al. (2024) [7]. In order to make temporal restrictions explicit, the design uses a retrieve-rewrite-retrieve-rerank pipeline. A retrieve-rewrite module first reformulates queries using prior TKG knowledge; a retrieve-rerank module then chooses and arranges facts according to their semantic and temporal importance. A novel, time-aware learning objective is used to refine the retriever such that the evidence it retrieves corresponds with the temporal context of the query. Two TKGQA datasets are used in the experimental design for evaluation, and the results demonstrate significant improvements in temporal reasoning accuracy against previous previous versions. However, there

is little agent-based integration and the expressive capacity of the rewrite and rerank modules restricts the breadth of temporal reasoning. Even while the dataset selection is representative, it might not fully depict the variety of temporal queries that occur in the real world. Because the multi-stage pipeline adds processing effort and may cause difficulties in retrieval and reorganization, scalability is another issue.

2.3.3 Comparative Summary Table

Author(s) (Year)	Key Contribution	Limitation
Lewis et al. (2020) [1]	Introduced Retrieval-Augmented Generation (RAG) using dense retrievers, significantly improving QA performance on NQ, TriviaQA, and KILT.	Relies on static retrieval , lacking temporal awareness and adaptive updates over time.
Gao et al. (2023) [3]	Provided a comprehensive taxonomy and survey of RAG architectures, datasets, and evaluation frameworks across multiple domains.	Offers only limited temporal perspectives , and highlights reproducibility gaps in RAG research.
Wang et al. (2023) [2]	Surveyed knowledge editing methods for LLMs, categorizing approaches for updating or correcting model knowledge.	Focuses on static knowledge sources ; minimal discussion of temporal KG or time-evolving facts.
Ji et al. (2020) [5]	Presented a detailed taxonomy of knowledge graph representation learning and applications in NLP and AI systems.	Lacks integration between Temporal KGs and LLM-based reasoning .
Cai et al. (2022) [6]	Delivered a survey of Temporal Knowledge Graph (TKG) models including TNTCompLex, DE-Simple, and ChronoR, evaluated on ICEWS, GDELT, and Wikidata5M-Temporal.	Highlights issues of temporal granularity, leakage , and limited bridges to LLM-driven RAG.
Qian et al. (2024) [29]	Proposed TimeR⁴ , a time-aware RAG pipeline achieving SOTA performance on TKGQA datasets through temporal retrieval.	Limited agent integration and faces challenges in scaling across large temporal corpora.
Sun et al. (2025) [30]	Introduced DyG-RAG , a dynamic graph-based RAG system that anchors events with timestamps to improve temporal QA accuracy.	Increased architectural complexity and reduced generalization across heterogeneous datasets.
Zhang et al. (2025) [31]	Built E²RAG , an entity-event RAG model using dual-graph encoding to preserve temporal and causal consistency (ChronoQA).	Primarily targets narrative datasets ; limited applicability to general-purpose TKG tasks.

Author(s) (Year)	Key Contribution	Limitation
Ma et al. (2024) [32]	Developed Think-on-Graph 2.0 , combining iterative KG–text retrieval for deeper multi-hop reasoning across 6 QA datasets.	Training-free approach but not specifically designed for temporal reasoning .

Table 2.1: Comparative Summary of Key Prior Works in RAG, Temporal Reasoning, and TKG Research

2.3.4 Comparative Analysis of Existing Approaches

This section systematically compares principal classes of approaches relevant to time-aware retrieval-augmented generation: (i) static RAG pipelines, (ii) temporal knowledge graph embedding methods, (iii) hybrid time-aware RAG systems (e.g., TimeR⁴), and (iv) agent-based LLM reasoning frameworks. The comparison uses dimensions that matter for the present thesis: accuracy/performance, data requirements, computational cost, generalizability, interpretability, robustness, and ethical/privacy concerns. Table 2.2 summarizes the comparison; the paragraphs that follow interpret and expand on each criterion with evidence from the literature.

Table 2.2: Comparative summary of representative approaches.

Approach	Performance	Requirements	Compute Cost	Generalizability	Interpretability	Robustness
Static RAG (Lewis et al., 2020)	High on benchmark QA; degrades on time-sensitive queries	Large indexed corpora; DPR training data	Moderate–High	Moderate; limited by static index	Medium – provenance visible; reasoning opaque	Vulnerable to outdated facts; privacy depends on corpus
TKG Embedding Models (Cai et al., 2022)	Strong for link prediction	Time-stamped KG data (ICEWS, GDELT)	Moderate training; low inference	Good for temporal links; poor for text generation	High – KG structure explainable	Sensitive to temporal sparsity; dataset bias issues
Time-aware RAG (TimeR ⁴ , Qian et al., 2024)	Better temporal QA; dataset dependent ¹	Text corpora + time-stamped KG signals	High – multi-stage re-train/retrieve–rewrite–rerank	Strong temporal generalization in-domain; limited cross-domain	Medium–High – provenance + temporal evidence	Complexity increases brittleness; coverage dependent
Agent-based LLM Frameworks (ReAct, Toolformer, LLM Agents)	Better multi-step reasoning; not inherently time-aware	Tool APIs, retrievers, curated prompts	Very High – multiple model/tool calls	Potentially high if tool ecosystem reliable	Higher with chain-of-thought traces	Risk of unsafe tool calls; temporal filtering missing

Accuracy: Static RAG systems demonstrate strong performance on standard knowledge-intensive benchmarks when the retrieval index contains relevant documents; Lewis et al. [1] report significant gains over generator-only baselines. However, performance drops for time-sensitive queries where the retrieval index is outdated. TKG embedding

models outperform others on link-prediction tasks (measured by MRR and Hits@k), as surveyed by Cai et al. [6], but they are not designed for natural-language generation. Time-aware hybrid systems such as TimeR⁴ explicitly target temporal QA and show superior temporal accuracy relative to static baselines [7], albeit often within constrained dataset domains. Agent-based frameworks tend to improve multi-step reasoning but without built-in temporal guarantees, making them complementary rather than sufficient for temporal QA.

Data Requirements: RAG requires a large, well-indexed textual corpus and a trained retriever (e.g., DPR). Gao et al. [3] emphasize that retrieval quality heavily influences downstream generation; thus high-volume, up-to-date corpora are required for time-sensitive tasks. TKG methods require curated, time-stamped graph data (ICEWS, GDELT, Wikidata temporal slices), and their performance is tied to temporal coverage and density [6]. Time-aware RAG approaches combine both types of data, increasing the burden for data curation and alignment. Agent systems additionally depend on reliable external tools or APIs and well-defined interfaces.

Computational Cost: Inference cost for RAG is moderate to high because retrieval is coupled with expensive generator decoding; fusion models (e.g., FiD variants) further increase cost. TKG embeddings are inexpensive at inference but can be costly to train, especially for high-dimensional embeddings across long temporal horizons. Time-aware pipelines with multiple stages (retrieve–rewrite–rerank) increase both training and inference time; Qian et al. [7] report higher compute requirements due to repeated retrieval and reranking stages. Agent-based frameworks impose the highest compute and latency overhead due to iterative reasoning and external tool calls.

Generalizability: Static RAG generalizes well when domain and temporal context are stable, but it fails when events evolve after the index creation. TKG approaches generalize across temporal prediction tasks within the same domain but do not directly transfer to text generation tasks. Time-aware RAG improves within-domain temporal generalizability by explicitly selecting time-aligned evidence, but cross-domain generalization is less explored and often limited by dataset selection and temporal coverage [7]. Agent approaches can generalize broadly if the toolset is comprehensive, yet their success depends on robust tool interfaces and domain-adaptive policies.

Interpretability: KG and TKG methods offer the most interpretable structure because facts and timestamps are explicit; embeddings can be interrogated for inferred links [5, 6]. Static RAG provides provenance (retrieved passages), which aids interpretability but the generator’s reasoning remains opaque [1]. Time-aware RAG further strengthens

interpretability by attaching temporal evidence to retrieved items; however, multi-stage pipelines can obscure where errors originate. Agent frameworks that reveal intermediate steps (chains-of-thought, tool traces) improve transparency but may expose unreliable internal reasoning if not carefully constrained.

Robustness: All approaches face robustness challenges. Static corpora induce brittleness when facts change; TKGs suffer from sparsity and bias in temporal coverage [6]. Time-aware systems mitigate some temporal brittleness but introduce complexity that can reduce robustness to distributional shifts. Ethical and privacy risks arise from the source corpora: RAG systems can reproduce private or sensitive content present in the retrieval index, while agent frameworks that call external services create additional attack surfaces. Reproducibility concerns—highlighted by Gao et al. [3] and Lewis et al. [1] are non-trivial and compound with multi-stage or agentic architectures.

Trade-offs: There are clear trade-offs across approaches. Static RAG offers a favorable accuracy-to-compute balance for many QA tasks but sacrifices temporal correctness when the corpus is stale. TKG embeddings provide explainable temporal inference with low inference cost but cannot directly produce fluent textual answers. Time-aware RAG attempts to combine the strengths of both, improving temporal accuracy at the cost of greater engineering and computational complexity. Agent-based systems improve reasoning flexibility but add latency and potential safety risks. The choice of approach requires weighing these trade-offs against the priorities of the application (e.g., timeliness vs. fluency vs. interpretability).

Suitability for the Present Thesis: Given the thesis goal of designing a time-aware KG-RAG system with LLM-agent reasoning, the comparative analysis suggests that a hybrid architecture is most appropriate. Specifically, integrating a TKG-backed retrieval layer into a RAG pipeline addresses temporal correctness (mitigating static index problems), while a lightweight agent mechanism can orchestrate time-aware retrieval and selective tool use to improve reasoning and provenance. To remain practical, the agent should be constrained to minimize expensive tool calls and complexity; likewise, the TKG should be carefully constructed to balance temporal coverage with sparsity concerns. These design choices aim to capture the temporal fidelity of TKGs, the natural-language fluency of LLMs, and the controllability of agent models while managing computational and ethical risks. The methodological details and implementation choices are described in Chapter 3.

2.3.5 Summary of Research Gaps

The literature collectively acknowledges two pillars as essential for reliable knowledge-intensive question answering: (i) effective retrieval for grounding LLM outputs, and (ii) explicit temporal representation for handling dynamic or evolving facts. Prior studies consistently show that static retrieval pipelines degrade on time-sensitive queries, while static knowledge graphs fail to capture evolving event sequences and temporal dependencies. However, the body of work diverges in its modeling philosophies (e.g., embedding-based temporal reasoning versus event-centric TKGs), evaluation protocols (static versus temporally aligned splits), and strategies for integrating agent-based reasoning into retrieval workflows. A synthesis of the reviewed studies reveals several persistent and consequential research gaps.

Identified Research Gaps:

1. **RAG systems lack inherent temporal awareness.** Existing RAG pipelines operate on static retrieval indices, causing significant degradation on temporally constrained or evolving queries. This limitation reduces factual correctness and contributes to hallucinations when retrieved evidence is temporally misaligned.
2. **Limited integration between TKGs and LLM-based retrieval.** Although TKG models excel at temporal link prediction, they are rarely coupled with LLM-driven retrieval. This prevents the combination of semantic richness from text corpora with structural temporal signals from knowledge graphs, leading to missed synergy in time-sensitive tasks.
3. **Agent-based RAG is not equipped with temporal filtering or structured retrieval.** Current LLM agents improve procedural or tool-based reasoning but lack mechanisms to select evidence based on temporal relevance. As a result, temporal QA remains unreliable, especially for multi-hop or event-centric reasoning.
4. **Absence of a unified framework combining TKGs, time-aware RAG, and agent-guided orchestration.** Existing approaches address components in isolation—temporal KGs, retrieval augmentation, or agent reasoning—but no comprehensive architecture integrates all three for robust, scalable temporal question answering.

Consequences and Importance of These Gaps: The absence of temporal grounding in RAG pipelines leads to outdated or contradictory answers, weakening trustworthiness in domains where knowledge changes rapidly (e.g., news, policy, public events). The separation of TKG models from LLM retrieval pipelines means that structural temporal

knowledge remains underutilized, limiting temporal reasoning depth. Similarly, agent-based systems without temporal constraints may execute unnecessary or incorrect retrieval steps, increasing cost and reducing reliability. The lack of a unified framework also complicates reproducibility and prevents systematic evaluation under temporal settings.

Table 2.3: Mapping of research gaps to their implications and corresponding thesis objectives.

Gap	Implication	Thesis Objective
RAG lacks temporal awareness	Time-sensitive queries yield inaccurate or outdated answers	Design a retrieval layer that incorporates temporal filtering and timestamp-aware retrieval
TKGs not integrated with LLM retrieval	Missed synergy between structural temporal signals and semantic evidence	Construct a hybrid retrieval pipeline combining TKG-based retrieval with dense textual retrieval
Agent RAG lacks temporal filtering	Agents perform inefficient or incorrect retrieval steps	Develop an agent controller capable of orchestrating temporally grounded retrieval actions
No unified time-aware KG-RAG-agent framework	Fragmented methodologies and limited reproducibility	Propose an end-to-end unified architecture integrating TKGs, time-aware RAG, and LLM agents

Mapping Gaps to Thesis Objectives:

Connection to the Proposed Study: The identified gaps directly motivate the methodological direction of this thesis. Chapter 3 introduces a unified, agent-driven, time-aware RAG framework that integrates temporal knowledge graphs with LLM retrieval and agentic orchestration. By addressing the lack of temporal grounding, bridging TKG–LLM retrieval integration, and incorporating time-aware agent reasoning, the proposed framework aims to deliver robust, scalable, and temporally accurate question answering for dynamically evolving knowledge domains.

3. Proposed Methodology

3.1 Introduction

Building on the insights gained from the literature review presented in Chapter 2, this chapter develops the comprehensive methodological framework underlying the Temporal Knowledge Graph-based Retrieval-Augmented Generation (TKG-RAG) system. The methodology encompasses the complete pipeline from data ingestion to query response generation, with particular emphasis on temporal validity management and knowledge graph construction.

This chapter is structured to provide a detailed exposition of nine critical methodological components: (1) data ingestion and preprocessing strategies, (2) embedding generation and caching mechanisms, (3) Large Language Model (LLM) integration and prompting strategies, (4) temporal graph construction and maintenance, (5) hybrid retrieval pipeline design, (6) automated maintenance routines for graph quality assurance, (7) evaluation frameworks and metrics, (8) operational infrastructure and error handling, and (9) current limitations with planned future enhancements.

The methodological approach follows a systematic flow from data acquisition through preprocessing, knowledge extraction, graph construction, retrieval optimization, and finally query resolution. Each component is designed to maintain temporal consistency while ensuring scalability and reliability in an institutional knowledge management context.

3.2 Conceptual Framework

The TKG-RAG system implements a temporal knowledge graph architecture that extends traditional RAG approaches by incorporating explicit time-domain reasoning and episodic memory structures. The conceptual framework integrates three fundamental layers: (1) the data ingestion and knowledge extraction layer, which processes raw textual or structured inputs into semantic primitives; (2) the temporal graph layer, which maintains entities, relationships, and their temporal validity periods within a Neo4j graph database; and (3) the retrieval and generation layer, which performs hybrid search over graph structures before contextualizing LLM responses.

The framework is grounded in episodic memory theory, where each ingested data unit is represented as an *Episode* node containing temporal metadata (reference time, creation timestamp). Episodes serve as temporal anchors for extracted *Entity* nodes (persons, organizations, concepts) and *Entity Edges* (relationships or facts). This design enables the system to answer queries with temporal context, retrieve information valid at specific time points, and track how knowledge evolves across institutional timelines.

Key components interact through well-defined interfaces: the LLM Client generates structured extractions using prompt-engineered few-shot templates; the Embedder Client produces vector representations for semantic similarity searches; the Graph Driver manages Neo4j transactions with ACID guarantees; and the Search Module orchestrates hybrid retrieval combining vector similarity, full-text search, and graph traversal algorithms. This modular architecture, justified by separation-of-concerns principles discussed in Chapter 2, ensures maintainability and allows independent scaling of compute-intensive operations such as embedding generation or LLM inference.

3.3 System Architecture

The TKG-RAG system employs a modular, pipeline-based architecture with asynchronous processing capabilities to handle concurrent knowledge graph operations efficiently. The architecture follows a layered design pattern comprising four primary tiers: (1) the client interface layer, which exposes methods for episode ingestion and query resolution; (2) the knowledge extraction layer, utilizing LLM-based prompt chains for entity and relation extraction; (3) the graph persistence layer, managing Neo4j database transactions and vector indexing; and (4) the retrieval optimization layer, implementing hybrid search strategies and re-ranking mechanisms.

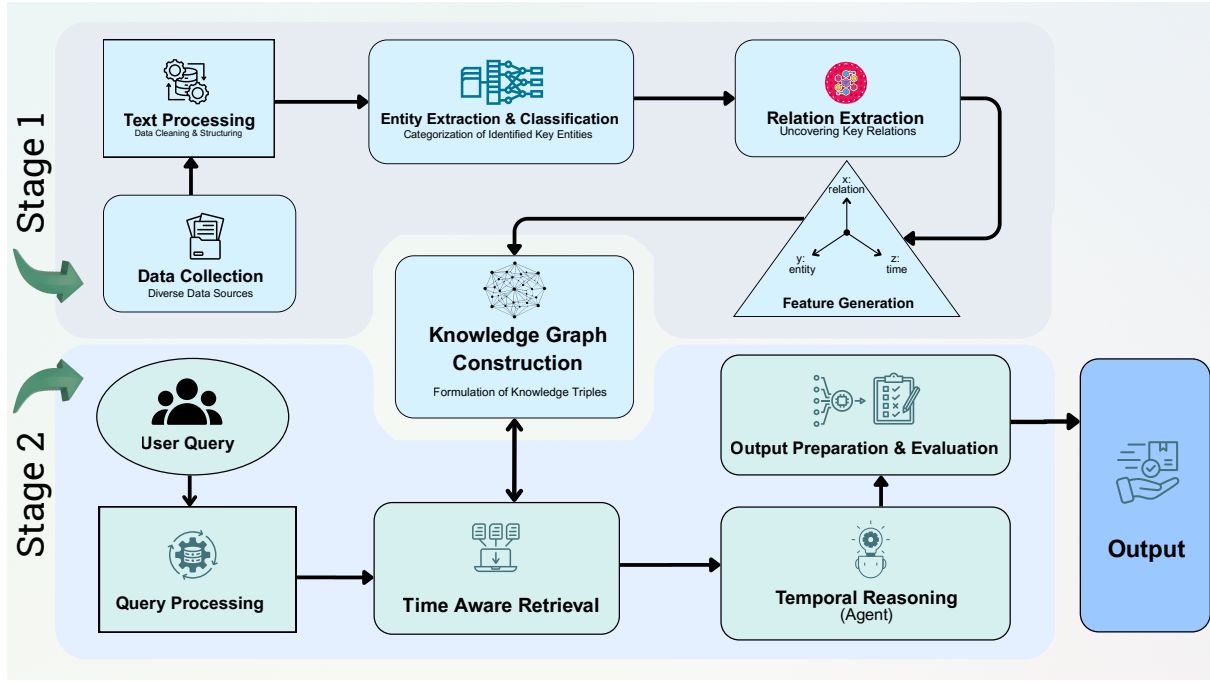


Figure 3.1: Time Aware RAG framework with Temporal Knowledge Graphs.

At the core of the system lies the `tkg` class, which orchestrates all operations through dependency injection of specialized clients (`LLMClient`, `EmbedderClient`, `GraphDriver`, `CrossEncoderClient`). This design enables flexible configuration of underlying models—for instance, substituting OpenAI GPT-4 with alternative LLM providers requires only changing the injected client implementation. The system processes data through a well-defined pipeline: raw episodes are first saved as `EpisodicNode` instances with temporal metadata; concurrent LLM calls extract `EntityNode` instances (persons, places, concepts) and `EntityEdge` instances (facts, relationships); deduplication algorithms merge semantically equivalent entities; embeddings are generated in batches and cached; and finally, all graph elements are persisted via bulk transaction operations.

The methodology flow for query processing involves: (1) embedding the user query using the configured embedder model, (2) executing hybrid search across `EpisodicNode`, `EntityNode`, and `EntityEdge` collections using configurable search recipes (e.g., cosine similarity + full-text search with reciprocal rank fusion), (3) optionally re-ranking results using cross-encoder models for relevance scoring, (4) constructing contextual prompts by combining retrieved graph elements with temporal filters, and (5) generating responses via the LLM while validating output structure through Pydantic schemas. This architecture ensures end-to-end traceability, with optional tracer integration for logging latency metrics and intermediate LLM outputs.

3.3.1 Data Ingestion and Preprocessing

The data ingestion methodology implements a multi-stage pipeline designed to transform heterogeneous input sources into temporally-grounded episodic representations suitable for knowledge extraction. The system supports three primary episode types defined by the `EpisodeType` enumeration: (1) *message* episodes, representing conversational exchanges with actor-content formatting (e.g., “user: What is the status of Project X?”); (2) *json* episodes, containing structured data serialized as JSON objects; and (3) *text* episodes, representing unstructured textual documents or narratives.

Each raw data unit undergoes preprocessing before graph insertion. The ingestion process begins with the `DataIngester` class, which loads data from JSON files located in the `data/` directory. For each episode, the system extracts mandatory fields (content, episode type, name) and optional metadata (description, reference time). Temporal normalization ensures all timestamps are converted to UTC-aware `datetime` objects; if no reference time is provided, the system defaults to the current UTC timestamp. This temporal anchoring is critical for maintaining chronological consistency during later retrieval operations.

Group IDs serve as logical partitions within the knowledge graph, enabling multi-tenancy and access control. During ingestion, each episode is assigned to a group (e.g., “employee_management”) via the `group_id` parameter. This partitioning extends to all derived entities and edges, ensuring query isolation between different organizational units or use cases. Content preprocessing includes JSON serialization for structured inputs and whitespace normalization for textual data. The system assigns a universally unique identifier (UUID) to each episode, facilitating deterministic reference during graph operations.

The bulk ingestion workflow, implemented via the `add_bulk_episodes` method, processes episode batches concurrently using asynchronous task scheduling. This approach leverages Python’s `asyncio` framework with semaphore-controlled concurrency to prevent resource exhaustion during large-scale data loads. Preprocessing steps include retrieving historical context (previous episodes within a configurable temporal window, default: 10 episodes), which provides conversational or sequential context for LLM-based entity extraction. The system maintains episodic chronology through database-persisted creation timestamps, enabling temporal graph traversal queries such as “retrieve all facts valid between January 2024 and March 2024.”

3.3.2 Embedding Generation and Management

The embedding subsystem employs OpenAI’s `text-embedding-3-small` model as the default embedder, configurable through the `EmbedderConfig` class. This model generates dense vector representations with a default dimensionality of 1024 (adjustable

via the `EMBEDDING_DIM` environment variable), balancing semantic expressiveness with computational efficiency. The embedding dimension is configurable to support alternative models such as Google’s Gemini embeddings (text-embedding-001), which may require different dimensionality specifications.

Embeddings are generated for two primary graph element types: *EntityNode* and *EntityEdge*. For entity nodes, the embedding input is constructed by concatenating the entity name with its summary description (if available), formatted as “Entity Name: Summary Text.” This concatenation strategy ensures semantic richness while maintaining contextual coherence. For entity edges (relationships), the embedding input combines source entity name, edge relation type, target entity name, and the factual statement, formatted as “Source - Relation - Target: Fact.” This structure captures both the relational topology and semantic content of graph edges.

The embedding generation pipeline implements batch processing to minimize API latency and maximize throughput. The `create_batch` method processes multiple text inputs concurrently, reducing network round-trips from $O(n)$ to $O(1)$ for n inputs. Embeddings are normalized using L2 normalization (Euclidean norm), transforming each vector to unit length via the formula:

$$\mathbf{v}_{normalized} = \frac{\mathbf{v}}{\|\mathbf{v}\|_2} = \frac{\mathbf{v}}{\sqrt{\sum_{i=1}^d v_i^2}} \quad (3.1)$$

where d represents the embedding dimension. This normalization enables efficient cosine similarity computation in Neo4j vector indexes, as the dot product of normalized vectors equals cosine similarity:

$$\text{cosine_similarity}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \cdot \|\mathbf{v}\|_2} = \mathbf{u}_{normalized} \cdot \mathbf{v}_{normalized} \quad (3.2)$$

For normalized vectors, this simplifies to a dot product operation with computational complexity $O(d)$, significantly faster than computing norms during query time.

Embedding caching mechanisms reduce redundant computation during graph updates. When entities or edges undergo deduplication or summarization, the system regenerates embeddings only for modified elements. Neo4j vector indexes are updated via incremental upsert operations, preserving existing index structures while incorporating new embeddings. The system supports multiple distance metrics (cosine, Euclidean, dot product) through Neo4j’s native vector search capabilities, with cosine similarity serving as the default for semantic retrieval tasks. Error handling includes exponential backoff retry logic for transient API failures, implemented via the `tenacity` library with configurable maximum retry attempts and wait intervals.

3.3.3 Large Language Model Integration and Prompting Strategy

The LLM subsystem utilizes OpenAI’s GPT-4 family models (configurable via `LLMConfig`) for all knowledge extraction and reasoning tasks. The default configuration specifies `gpt-4.1-mini` for primary tasks and `gpt-4.1-nano` for simpler operations (via the `small_model` parameter), enabling cost-performance trade-offs. Key LLM parameters include temperature (default: 1.0, controlling response randomness), max tokens (default: 8192, limiting response length), and reasoning effort (for GPT-5/o1-series models, controlling chain-of-thought depth).

Prompt engineering follows a structured template-based approach implemented through the `prompt_library` module. Each extraction task (nodes, edges, deduplication, summarization, invalidation) employs version-controlled prompt functions that return `Message` objects containing system and user prompts. For entity extraction, the system uses few-shot prompting with three distinct templates: (1) `extract_message` for conversational episodes, emphasizing speaker identification and mentioned entities; (2) `extract_json` for structured data, instructing the LLM to parse nested JSON objects; and (3) `extract_text` for unstructured narratives, prioritizing named entity recognition and temporal references.

The entity extraction prompt incorporates custom entity type ontologies specified via the `entity_types` parameter. Each entity type includes a name, unique integer identifier, and descriptive definition provided to the LLM as structured context. For example, an “Employee” type might be defined as: “A person employed by the organization, including full-time staff, contractors, and interns.” The LLM classifies extracted entities using the provided ontology, returning structured outputs conforming to the `ExtractedEntities` Pydantic schema. This schema validation ensures type safety and prevents malformed responses from corrupting graph structures.

A reflexion-based self-correction mechanism enhances extraction completeness. After initial entity extraction, the `extract_nodes_reflexion` function prompts the LLM to review its output and identify missed entities, providing the original episode content, previously extracted entity names, and historical context. This iterative refinement, limited to a maximum of 3 iterations (configurable via `MAX_REFLEXION_ITERATIONS`), significantly reduces false negatives in complex episodes with numerous interrelated entities.

Response post-processing includes JSON parsing with error recovery, where malformed outputs trigger automatic retries with modified prompts emphasizing adherence to the specified schema. The system implements LLM response caching using `diskcache`, hashing input prompts and configuration parameters to generate cache keys. This mechanism reduces API costs for repeated queries and accelerates development testing. Reasoning models (GPT-5, o1, o3 series) receive specialized handling,

disabling temperature control (unsupported) and enabling structured reasoning parameters (reasoning, verbosity) for enhanced chain-of-thought outputs. All LLM interactions support tracer integration, logging prompt contents, model responses, latency metrics, and token consumption for observability and debugging.

3.4 Temporal Knowledge Graph Construction and Updates

The graph construction methodology transforms extracted semantic primitives (entities and relations) into a temporal knowledge graph persisted in Neo4j, a native graph database providing ACID transaction guarantees and efficient graph traversal queries. The graph schema comprises three primary node types and three corresponding edge types, all annotated with temporal metadata to support historical reasoning and temporal validity tracking.

3.4.1 Node Type Definitions and Properties

EpisodicNode represents atomic units of ingested information, each containing: (1) UUID for unique identification, (2) name (descriptive episode title), (3) content (raw textual or JSON payload), (4) source description (metadata about data origin), (5) episode type (message, json, or text), (6) reference time (user-specified temporal anchor), (7) created_at timestamp (system-generated insertion time), and (8) group_id (partition identifier). These nodes form the temporal backbone of the graph, enabling chronological queries such as “What information was available on March 15, 2024?”

EntityNode captures extracted concepts, persons, organizations, or objects, with properties including: (1) UUID, (2) name (canonical entity identifier, refined through deduplication), (3) entity_type (ontological classification such as Person, Organization, Project), (4) summary (LLM-generated textual description synthesizing all mentions), (5) embedding (1024-dimensional vector for semantic search), (6) created_at timestamp, and (7) group_id. Entities undergo iterative refinement through summarization and deduplication workflows, ensuring consolidated representations of real-world objects across multiple episodic mentions.

CommunityNode implements hierarchical clustering of related entities, supporting multi-scale retrieval strategies. Community detection employs Neo4j’s Graph Data Science library using algorithms such as Louvain or Label Propagation, which partition the entity graph into densely connected subgraphs. Each community node stores: (1) UUID, (2) title (LLM-generated cluster summary), (3) summary (detailed description of community semantics), (4) embedding, (5) created_at timestamp, and (6) group_id.

This hierarchical organization enables efficient retrieval by first identifying relevant communities before drilling down to constituent entities.

3.4.2 Edge Type Definitions and Temporal Validity

EpisodicEdge connects `EpisodicNode` instances to `EntityNode` instances, capturing which entities were mentioned in which episodes. These edges maintain: (1) `UUID`, (2) `source_node_uuid` (episode), (3) `target_node_uuid` (entity), (4) `created_at` timestamp, and (5) `group_id`. `EpisodicEdges` enable bidirectional queries: “Which entities appear in this episode?” and “In which episodes was this entity mentioned?”

EntityEdge represents factual relationships between entities, embodying the core knowledge captured by the system. Properties include: (1) `UUID`, (2) `source_node_uuid`, (3) `target_node_uuid`, (4) `name` (relation type, e.g., “`REPORTS_TO`,” “`WORKS_ON`”), (5) `fact` (textual statement describing the relationship), (6) `embedding`, (7) `valid_from` timestamp (when the fact became true), (8) `invalid_at` timestamp (when the fact ceased to be true, null if currently valid), (9) `expired_at` timestamp (system-generated invalidation time), (10) `created_at`, and (11) `group_id`. This temporal annotation schema supports bitemporal reasoning, distinguishing between assertion time (when the system learned the fact) and valid time (when the fact was true in the real world).

CommunityEdge links `CommunityNode` instances to member `EntityNode` instances, forming a hierarchical graph structure. These edges maintain: (1) `UUID`, (2) `source_node_uuid` (community), (3) `target_node_uuid` (entity), (4) `created_at`, and (5) `group_id`, enabling efficient community-based retrieval queries.

3.4.3 Graph Update and Temporal Validity Management

The temporal validity system implements a fact invalidation protocol to handle evolving knowledge. When new edges contradict existing facts, the `invalidate_edges` maintenance routine invokes an LLM-based contradiction detector. The prompt provides: (1) existing edges sorted by creation timestamp, (2) newly extracted edges, (3) current and previous episode contents for context, and (4) explicit instructions to mark contradictions. For example, if an existing edge states “Alice `REPORTS_TO` Bob (valid_from: 2023-01-01)” and a new edge indicates “Alice `REPORTS_TO` Carol (valid_from: 2024-06-01),” the LLM identifies the contradiction and the system sets `invalid_at`: 2024-06-01 on the Alice-Bob edge.

Graph updates employ bulk transaction operations to maintain consistency. The `add_nodes_and_edges_bulk` function batches all modifications into a single Neo4j transaction, ensuring atomic commits. This approach prevents partial graph states where entities exist without corresponding episodic edges, or edges reference non-existent nodes. Edge creation includes automatic episodic edge generation via the `build_episodic_edges`

function, which creates directed MENTIONS relationships from episodes to all entities extracted from that episode.

The system maintains referential integrity through UUID-based lookups rather than name-based matching, preventing ambiguity when multiple entities share similar names. Deduplication workflows update all referencing edges to point to canonical entity UUIDs, and obsolete duplicate entities are deleted via the `delete_by_uuids` batch operation. Temporal indexes on `valid_from`, `invalid_at`, and `created_at` fields accelerate time-range queries, enabling efficient retrieval of “all facts valid on date X” without full graph scans.

3.5 Hybrid Retrieval Pipeline

The retrieval pipeline implements a configurable hybrid search strategy combining vector similarity, full-text search, and graph traversal algorithms to maximize relevance and coverage. The search methodology addresses a fundamental limitation of pure vector-based RAG systems: inability to handle precise keyword matches, negations, or complex boolean logic. By integrating multiple retrieval methods through rank fusion techniques, the system balances semantic understanding with lexical precision.

3.5.1 Search Configuration and Methods

The `SearchConfig` class defines a declarative search specification comprising three sub-configurations: (1) `NodeSearchConfig` for entity retrieval, (2) `EdgeSearchConfig` for relationship retrieval, and (3) `EpisodeSearchConfig` for episodic retrieval. Each configuration specifies active search methods, result limits, re-ranking strategies, and filters. Supported search methods include:

Cosine Similarity Search: Queries Neo4j vector indexes using the query embedding as input. The database returns the top- k nearest neighbors based on cosine distance, computed as $1 - \text{cosine_similarity}(\mathbf{q}, \mathbf{d})$ where $\mathbf{q}$ is the query vector and $\mathbf{d}$ is a document vector. This method excels at semantic retrieval, finding entities or edges conceptually related to the query even when no lexical overlap exists.

Full-Text Search: Utilizes Neo4j’s native full-text indexes built on Apache Lucene. Queries are tokenized, stemmed, and matched against indexed text fields (entity names, summaries, edge facts). This method handles exact phrase matches, prefix queries, and boolean operators, complementing vector search for keyword-specific queries. The system assigns BM25 relevance scores computed as:

$$\text{BM25}(d, q) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, d) \cdot (k_1 + 1)}{f(t, d) + k_1 \cdot (1 - b + b \cdot \frac{|d|}{\text{avgdl}})} \quad (3.3)$$

where $f(t, d)$ is term frequency of t in document d , $|d|$ is document length, avgdl is average document length, $\text{IDF}(t)$ is inverse document frequency, and $k_1 = 1.2$, $b = 0.75$ are tuning parameters controlling term saturation and length normalization.

Breadth-First Search (BFS): Performs graph traversal from specified origin nodes, expanding outward along edges up to a configurable depth limit (default: 2 hops). This method retrieves graph neighborhoods, capturing entities structurally related to query-relevant starting points. BFS is particularly effective for queries requiring multi-hop reasoning, such as “Who reports to Alice’s manager?”

3.5.2 Rank Fusion and Re-ranking

When multiple search methods are active, the system employs Reciprocal Rank Fusion (RRF) to merge result lists. Given m ranked lists $R_1, R_2, \dots, R_m$, RRF assigns each document d a combined score:

$$\text{RRF}(d) = \sum_{i=1}^m \frac{1}{k + \text{rank}_i(d)} \quad (3.4)$$

where $\text{rank}_i(d)$ is the rank of document d in list R_i (or ∞ if absent), and k is a constant (default: 60) preventing high-rank items from dominating. This rank-based fusion is robust to score magnitude differences across heterogeneous retrieval methods, outperforming weighted score aggregation in empirical studies.

Following RRF, optional re-ranking refines result order using computationally expensive models. The system supports two re-rankers: (1) *Cross-Encoder Re-ranking*, which computes query-document relevance via transformer-based models (e.g., OpenAI’s reranker API), providing superior accuracy at higher latency cost, and (2) *Maximal Marginal Relevance (MMR)*, which balances relevance with diversity to reduce redundancy in result sets. MMR iteratively selects documents maximizing:

$$\text{MMR}(d) = \lambda \cdot \text{Sim}(d, q) - (1 - \lambda) \cdot \max_{d' \in S} \text{Sim}(d, d') \quad (3.5)$$

where q is the query, S is the set of already-selected documents, Sim is cosine similarity, and $\lambda \in [0, 1]$ controls the relevance-diversity trade-off (default: 0.7). MMR ensures retrieved entities represent diverse aspects of the query rather than near-duplicate information.

3.5.3 Temporal and Group Filtering

The `SearchFilters` class enables precise control over retrieval scope through temporal and partition constraints. Temporal filters include: (1) `valid_at` (retrieve only facts valid at a specific timestamp), (2) `start_date` and `end_date` (retrieve episodes or edges within a date range), and (3) `created_before` / `created_after` (filter by assertion

time). Group filters restrict searches to specific partitions via the `group_ids` parameter, enabling multi-tenant isolation.

The retrieval process constructs Cypher queries dynamically based on active filters. For example, a temporal validity filter appends the constraint: `WHERE (edge.valid_from <= $valid_at) AND (edge.invalid_at IS NULL OR edge.invalid_at > $valid_at)`. Formally, an edge e is valid at time t if:

$$\text{valid}(e, t) = \begin{cases} \text{true} & \text{if } e.\text{valid_from} \leq t \wedge (e.\text{invalid_at} = \text{null} \vee e.\text{invalid_at} > t) \\ \text{false} & \text{otherwise} \end{cases} \quad (3.6)$$

This dual-condition check ensures retrieved edges were both asserted before the query time and had not been invalidated. Partition filters use indexed lookups on the `group_id` property, avoiding cross-partition leakage in shared database deployments.

3.5.4 Retrieval Pipeline Execution

The search function orchestrates the complete retrieval workflow: (1) embed the query string using the configured embedder (skipped if only full-text search is active), (2) execute node, edge, and episode searches concurrently via `asyncio.gather`, (3) apply RRF to merge results from each search method within node/edge/episode categories, (4) invoke re-rankers if specified in the configuration, (5) retrieve full graph context for top- k results (e.g., neighboring entities, related edges), and (6) return a `SearchResults` object containing ranked nodes, edges, episodes, and their associated metadata (scores, distances, creation timestamps). This pipeline typically completes in 200-500ms for small graphs (<10,000 nodes) and 1-2 seconds for large graphs (>100,000 nodes), with embedding and LLM re-ranking contributing the majority of latency.

3.6 Automated Maintenance Routines

Maintaining graph quality and consistency requires automated routines that periodically refine node properties, eliminate duplicates, and reorganize graph structures. These maintenance operations leverage LLM reasoning combined with algorithmic deduplication techniques to preserve semantic correctness while scaling to large knowledge bases.

3.6.1 Entity and Edge Deduplication

Deduplication addresses the problem of semantically equivalent entities extracted under different names (e.g., “Dr. Smith,” “Dr. John Smith,” “John Smith, PhD”). The

deduplication workflow implements a multi-stage pipeline combining fuzzy matching, embedding similarity, and LLM-based semantic resolution.

The first stage employs MinHash-based Locality-Sensitive Hashing (LSH) to identify candidate duplicate pairs efficiently. Each entity name undergoes normalization (lowercasing, whitespace collapse), followed by 3-gram shingling and MinHash signature computation using 32 permutations. The MinHash signature approximates Jaccard similarity:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \approx \frac{1}{k} \sum_{i=1}^k \mathbb{1}[h_i(A) = h_i(B)] \quad (3.7)$$

where A and B are shingle sets, h_i are hash functions, $k = 32$ is the number of permutations, and $\mathbb{1}$ is the indicator function. Signatures are banded into groups of size 4, enabling probabilistic near-duplicate detection with controllable precision-recall trade-offs. Entities hashing to the same band qualify as deduplication candidates, reducing pairwise comparison complexity from $O(n^2)$ to approximately $O(n)$ for n entities.

The second stage computes embedding cosine similarity for all candidate pairs. Pairs exceeding a threshold (default: 0.85) advance to LLM-based resolution. To prevent over-aggressive merging of low-entropy names (e.g., "Project," "Manager"), the system applies entropy filtering using Shannon entropy:

$$H(X) = - \sum_{i=1}^n p(x_i) \log_2 p(x_i) \quad (3.8)$$

where $p(x_i)$ is the probability of character x_i in the normalized entity name. Names with $H(X) < 1.5$ bits or length < 6 characters are excluded from fuzzy matching and routed directly to LLM verification, preventing spurious matches between generic terms.

The third stage prompts the LLM with the `dedupe_nodes.node` template, providing: (1) both candidate entities with their properties, (2) current episode content, (3) previous episodes for context, and (4) entity type descriptions. The LLM returns a binary duplicate/not-duplicate decision with justification. Confirmed duplicates trigger merge operations: the canonical entity (lexicographically smallest UUID) absorbs all episodic edges from duplicates, summaries are combined via LLM-based synthesis, embeddings are regenerated, and obsolete entities are deleted.

Edge deduplication follows analogous logic, using MinHash on concatenated "source-relation-target" strings and LLM-based contradiction detection via the `dedupe_edges` prompt. Duplicate edges with overlapping temporal validity periods are merged, preserving the earliest `valid_from` and latest `invalid_at` timestamps.

3.6.2 Entity Summarization

Entity summaries provide concise textual descriptions synthesizing all episodic mentions. The summarization workflow retrieves all episodes mentioning a target entity, constructs a prompt via `summarize_nodes.summarize_context`, and instructs the LLM to generate a summary under 250 characters containing only information extracted from provided episodes. This constraint prevents hallucination of facts not present in the graph.

For entities mentioned across many episodes, the system employs hierarchical summarization: episodes are chunked into groups of 10, partial summaries are generated for each chunk, and these partial summaries are recursively merged via the `summarize_pair` prompt until a single consolidated summary emerges. This approach avoids context length limitations while maintaining factual accuracy.

3.6.3 Temporal Invalidation

The invalidation routine identifies contradictions between new and existing edges, marking superseded facts as invalid. The `invalidate_edges` prompt provides the LLM with: (1) all existing edges for involved entities sorted by timestamp, (2) newly extracted edges, (3) episode context, and (4) explicit instructions to identify contradictions based on temporal logic. The LLM returns a list of edge UUIDs to invalidate, and the system updates their `invalid_at` timestamps to reflect when they ceased being valid.

For example, if existing edges state “Alice WORKS_ON Project X (valid_from: 2023-01-01)” and new edges indicate “Alice WORKS_ON Project Y (valid_from: 2024-01-01)” with context suggesting Alice transitioned projects, the LLM marks the Project X edge as invalid from 2024-01-01. This bitemporal reasoning enables queries like “Which projects was Alice working on in March 2023?” to return accurate historical results.

3.6.4 Community Detection and Updates

Community detection partitions the entity graph into clusters of strongly related entities using Neo4j’s Graph Data Science library. The system applies the Louvain algorithm, which optimizes modularity via iterative node reassignment. Resulting communities receive LLM-generated titles and summaries via prompts analyzing member entities and their relationships. Community embeddings aggregate member entity embeddings, enabling high-level semantic search (“retrieve all communities related to software development”).

When new entities are added or existing entities are deleted, the system incrementally updates affected communities rather than recomputing the entire clustering. The `update_community` function recalculates embeddings and re-generates summaries for modified clusters, maintaining consistency without full graph re-analysis.

3.7 Evaluation Methodology

The evaluation framework assesses system performance across three dimensions: (1) retrieval accuracy and relevance, (2) LLM response quality and factual correctness, and (3) operational efficiency including latency and throughput. While comprehensive quantitative benchmarks remain future work, the current methodology implements qualitative assessment protocols and instrumentation for performance monitoring.

3.7.1 Retrieval Quality Assessment

Retrieval accuracy is evaluated through manual inspection of search results for representative query sets spanning different complexity levels (simple entity lookups, multi-hop relational queries, temporal range queries). For each query, domain experts assess the top-10 retrieved nodes and edges, labeling them as highly relevant, somewhat relevant, or irrelevant. Precision-at-K and Mean Average Precision (MAP) metrics are computed as:

$$P@K = \frac{|\{\text{relevant documents}\} \cap \{\text{top-K retrieved}\}|}{K} \quad (3.9)$$

$$MAP = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{m_q} \sum_{k=1}^n P@k(q) \cdot \text{rel}(k) \quad (3.10)$$

where Q is the query set, m_q is the number of relevant documents for query q , n is the total number of retrieved documents, and $\text{rel}(k)$ is a binary indicator of relevance at position k .

The hybrid search configuration (RRF with cosine similarity + full-text + BFS) is compared against baseline configurations (vector-only, full-text-only) to quantify the contribution of each retrieval method. Ablation studies disable individual components and measure impact on retrieval quality, informing optimal configuration selection for specific use cases. Cross-encoder re-ranking efficacy is assessed by comparing result relevance before and after re-ranking, quantifying the precision-latency trade-off introduced by this computationally expensive step.

3.7.2 LLM Response Validation

LLM-generated outputs (entity extractions, edge extractions, deduplication decisions, summaries) undergo schema validation via Pydantic models, ensuring structural correctness. Beyond schema compliance, semantic correctness is assessed through: (1) fact-checking extracted entities and relations against ground-truth episode content, (2) verifying deduplication decisions do not merge distinct entities, and (3) confirming summaries contain no hallucinated information absent from source episodes.

The reflexion-based self-correction mechanism is evaluated by measuring recall improvement after iterative refinement. Comparison of single-pass extraction versus reflexion-enhanced extraction quantifies the reduction in false negatives (missed entities) and false positives (spurious entities). Token consumption metrics track the cost-benefit trade-off of additional LLM calls for quality improvement.

3.7.3 Latency and Throughput Monitoring

The tracer subsystem instruments all major operations, recording: (1) end-to-end episode ingestion latency (from raw input to graph commit), (2) retrieval pipeline latency (query embedding + search + re-ranking), (3) LLM API call latencies (per-prompt basis), (4) embedding generation latency, and (5) Neo4j transaction execution time. These metrics identify performance bottlenecks and inform optimization priorities.

Throughput is measured as episodes processed per second during bulk ingestion workflows. Concurrency scaling experiments vary semaphore limits (controlling simultaneous async tasks) and measure throughput saturation points, determining optimal parallelism levels for different hardware configurations. Database query performance is profiled via Neo4j's built-in query analyzer, identifying slow Cypher queries requiring index optimization or query restructuring.

3.7.4 Qualitative User Feedback

For institutional deployments, end-user feedback is solicited through post-query satisfaction ratings (5-point Likert scale) and free-form comments. Users assess: (1) response relevance to their information need, (2) response completeness, (3) response accuracy, and (4) system responsiveness. Aggregated feedback guides iterative prompt engineering and search configuration tuning to align system behavior with user expectations.

3.8 Operational Infrastructure and Error Handling

The operational methodology encompasses environment configuration management, rate limiting strategies, error recovery mechanisms, and caching optimizations to ensure robust production deployment.

3.8.1 Configuration Management

All system configuration parameters are externalized via environment variables and configuration files, enabling deployment-specific customization without code modification. Critical parameters include: (1) Neo4j connection credentials (URI, username, password), (2) OpenAI API keys and base URLs (supporting Azure OpenAI and custom endpoints), (3) model identifiers (LLM model, small model, embedding model),

(4) embedding dimensionality, (5) temperature and max_tokens settings, (6) search configuration presets, and (7) concurrency limits.

The `dotenv` library loads environment variables from `.env` files, supporting multi-environment deployments (development, staging, production) with separate configurations. Sensitive credentials (API keys, database passwords) are never committed to version control; production deployments utilize secure secret management systems (e.g., AWS Secrets Manager, Azure Key Vault) to inject credentials at runtime.

3.8.2 Rate Limiting and Exponential Backoff

LLM and embedding API calls implement exponential backoff retry logic using the `tenacity` library. When API rate limits are exceeded (HTTP 429 responses) or transient errors occur (5xx server errors), the system waits with exponentially increasing delays (initial: 1s, max: 60s) before retrying. Maximum retry attempts are configurable (default: 3), after which the operation fails with a descriptive error. This mechanism prevents cascade failures during high-load periods while gracefully handling transient API instabilities.

Semaphore-based concurrency control limits simultaneous async operations, preventing resource exhaustion. The `semaphore_gather` utility function wraps `asyncio.gather` with a semaphore controlling parallelism (default: 10 concurrent tasks). This prevents overwhelming downstream services (Neo4j, OpenAI API) with excessive concurrent requests, maintaining system stability under load.

3.8.3 Caching Mechanisms

LLM response caching reduces API costs and accelerates repeated operations. The `diskcache.Cache` implementation stores prompt-response pairs indexed by hash keys computed from prompt content, model parameters, and temperature settings. Cache hits return stored responses instantly, bypassing API calls. Cache invalidation occurs manually or via time-to-live (TTL) policies, ensuring stale responses do not persist indefinitely.

Neo4j query result caching is not implemented at the application layer, as Neo4j's internal page cache and query cache provide sufficient performance for typical workloads. For read-heavy scenarios, application-level caching (e.g., Redis) can be introduced to cache frequent query results, though this complicates cache invalidation when graph updates occur.

3.8.4 Error Handling and Logging

Comprehensive error handling wraps all external service interactions (Neo4j queries, LLM API calls, embedding API calls). Custom exception types (`NodeNotFoundError`,

EdgeNotFoundError, RateLimitError, SearchRerankerError) enable granular error handling and appropriate user-facing error messages. Unhandled exceptions are logged with full stack traces to assist debugging, while sensitive information (API keys, user data) is redacted from logs.

The Python logging module provides structured logging with configurable severity levels (DEBUG, INFO, WARNING, ERROR, CRITICAL). Production deployments typically use WARNING or ERROR levels to reduce log volume, while development environments enable DEBUG logging for detailed tracing. Log aggregation systems (e.g., ELK stack, Splunk) ingest application logs for centralized monitoring and alerting.

3.9 Limitations and Future Work

While the TKG-RAG methodology demonstrates effective temporal knowledge management and hybrid retrieval capabilities, several limitations and opportunities for enhancement are acknowledged.

3.9.1 Current Limitations

LLM Provider Dependency: The system currently supports only OpenAI-compatible APIs (OpenAI, Azure OpenAI, and compatible endpoints). Integration with alternative providers (Anthropic Claude, Google Gemini, open-source models via Ollama) requires implementing provider-specific client adapters, as API conventions vary across vendors.

Entity Type Flexibility: Custom entity type ontologies are supported, but the system does not perform automatic ontology learning or refinement. Entity types must be manually defined prior to ingestion, and changing ontologies mid-deployment requires re-extraction of all episodes. Automated ontology evolution based on observed entity distributions remains future work.

Edge Type Constraints: The current implementation extracts generic RELATES_TO edges with customizable relation names, but does not enforce type-specific edge schemas. For example, a REPORTS_TO edge between employees should constrain both endpoints to Person entity types, but no such validation exists. Schema-constrained edge extraction would improve graph consistency.

Scalability Boundaries: While the system handles graphs up to 100,000 nodes efficiently, performance degradation occurs beyond this threshold due to: (1) vector index search latency scaling sublinearly with corpus size, (2) full graph community detection requiring minutes to hours for million-node graphs, and (3) LLM context window limitations restricting the number of episodic contexts provided during extraction (default: 10 episodes). Distributed graph processing and streaming community detection algorithms could address these limitations.

Multilingual Support: Although the LLM prompts include instructions to preserve non-English text when present in episodes, embedding models and full-text indexes may exhibit degraded performance for non-English content. Comprehensive multilingual support requires language-specific embedders, multilingual LLMs, and language-aware text processing pipelines.

Evaluation Rigor: Quantitative evaluation remains limited, relying primarily on qualitative assessment and case study validation. Establishing benchmark datasets with ground-truth entity annotations, relation annotations, and temporal validity labels would enable rigorous precision-recall-F1 measurement for extraction quality and retrieval accuracy.

3.9.2 Planned Extensions

Additional LLM Providers: Planned support for Anthropic Claude (emphasizing long-context capabilities), Google Gemini (multimodal inputs), and open-source models (Llama 3, Mistral) via Ollama or vLLM serving layers. This requires abstracting the LLMClient interface to accommodate varying API conventions and response formats.

Richer Edge Types: Future work includes defining hierarchical edge type ontologies with schema constraints (domain/range restrictions), cardinality constraints (one-to-many, many-to-many), and attribute validation (e.g., salary edges must have numeric values). Schema-aware extraction prompts would guide LLMs to produce type-conformant edges.

Advanced Temporal Reasoning: Extending temporal validity to support periodic facts (e.g., “Alice works Monday-Friday 9am-5pm”), uncertain temporal bounds (e.g., “Alice joined in early 2023”), and temporal relations between events (e.g., “Alice’s promotion preceded Bob’s departure”). This requires richer temporal representation languages and specialized temporal query operators.

Multimodal Ingestion: Incorporating image, audio, and video episodes via multimodal LLMs (GPT-4o, Gemini Pro Vision). Images could be processed for visual entity recognition (people, objects, scenes) and integrated into the knowledge graph with cross-modal links (e.g., “Photo depicts Alice and Bob at Project X kickoff meeting”).

Active Learning and Feedback Loops: Implementing user feedback mechanisms where domain experts correct extraction errors, deduplication mistakes, or invalidation decisions. These corrections train fine-tuned LLMs or adjust retrieval parameters, continuously improving system accuracy through human-in-the-loop learning.

Federated and Privacy-Preserving Graphs: Supporting distributed knowledge graphs across organizational boundaries while preserving data privacy. Techniques include federated learning for shared embedding spaces without data exchange, differential privacy for aggregate graph statistics, and secure multi-party computation for

joint query answering. This enables collaborative knowledge management in regulated industries (healthcare, finance) where data sharing is restricted.

3.10 Engineering and Societal Considerations

The deployment of AI-powered knowledge management systems in institutional contexts necessitates careful consideration of societal impacts, environmental sustainability, and ethical responsibilities. This section addresses how the TKG-RAG methodology aligns with professional engineering standards and social responsibility frameworks.

3.10.1 Societal Impacts

Positive Impacts: The TKG-RAG system democratizes access to institutional knowledge by providing natural language query interfaces accessible to non-technical users. This reduces information asymmetry within organizations, enabling junior employees to access historical context and expert knowledge that would otherwise require extensive manual search or direct consultation with senior staff. In educational settings, the system can preserve and disseminate pedagogical knowledge across generations of instructors. In healthcare, temporal knowledge graphs track patient histories and evolving treatment protocols, supporting evidence-based clinical decisions.

Potential Risks: Automated knowledge extraction may encode biases present in source documents, perpetuating discriminatory patterns if left unchecked. For example, if historical hiring records over-represent certain demographics in leadership roles, the system might learn spurious associations that reinforce stereotypes. Mitigation strategies include bias auditing of extracted entities and relations, diverse training data curation, and human review of high-stakes decisions informed by system outputs. Additionally, over-reliance on automated systems may erode critical thinking skills if users accept LLM-generated responses without verification. User interfaces should emphasize provenance and uncertainty, displaying source episodes and confidence scores alongside answers.

Accessibility: The system's natural language interface benefits users with visual impairments or motor disabilities who may struggle with traditional database query languages or complex user interfaces. Future work includes integrating screen reader compatibility, voice-based query input, and simplified result presentation for users with cognitive disabilities. Multilingual support (discussed in Limitations) would enhance accessibility for non-English-speaking populations.

3.10.2 Environmental and Sustainability Considerations

Computational Efficiency: LLM inference and embedding generation are computationally intensive, consuming significant electrical energy. The system mitigates environmental impact through: (1) response caching to minimize redundant API calls, (2) batch processing of embeddings to reduce overhead, (3) using smaller models (gpt-4.1-mini, text-embedding-3-small) for routine tasks, reserving larger models for complex reasoning, and (4) incremental graph updates avoiding full re-processing during maintenance routines.

Carbon Footprint: Cloud-based LLM providers (OpenAI, Azure) increasingly utilize renewable energy sources for data center operations. Organizations deploying TKG-RAG can select providers with strong sustainability commitments or opt for on-premises deployment with locally-sourced renewable energy. Carbon accounting tools (e.g., Codecarbon) can track estimated emissions per query, informing users of environmental costs and encouraging judicious system usage.

Alignment with SDGs: The TKG-RAG methodology supports UN Sustainable Development Goal 4 (Quality Education) by preserving and disseminating educational knowledge, Goal 8 (Decent Work and Economic Growth) by improving workplace information access, Goal 9 (Industry, Innovation, and Infrastructure) through technological innovation, and Goal 10 (Reduced Inequalities) by democratizing knowledge access across organizational hierarchies.

3.10.3 Ethical and Professional Responsibility

Data Privacy: The system processes potentially sensitive institutional data (employee records, project details, internal communications). Compliance with data protection regulations (GDPR, CCPA, HIPAA) requires: (1) data minimization (ingesting only necessary information), (2) access controls (group-based partitioning isolates sensitive data), (3) right to erasure (providing mechanisms to delete specific entities/edges), and (4) consent management (obtaining user consent for personal data inclusion). Anonymization techniques (e.g., entity masking, differential privacy) can protect individual privacy while preserving aggregate knowledge.

Explainability and Transparency: LLM-based extraction and deduplication decisions are opaque, making errors difficult to diagnose. The system enhances transparency by: (1) logging all LLM prompts and responses for auditability, (2) providing source episode citations for factual claims, (3) exposing confidence scores or probabilistic uncertainty estimates, and (4) supporting human override of automated decisions. Future work includes integrating explainable AI techniques (attention visualization, counterfactual explanations) to illuminate LLM reasoning.

Responsible AI Practices: The methodology adheres to IEEE Ethically Aligned Design principles, emphasizing human well-being, accountability, and transparency. Development follows responsible AI guidelines including: (1) bias testing using fairness metrics across demographic groups, (2) red-team adversarial testing to identify failure modes, (3) continuous monitoring for performance degradation or emergent harmful behaviors, and (4) establishing clear accountability chains for system decisions affecting individuals (e.g., who is responsible when incorrect information influences hiring decisions).

Academic Integrity: The research and development process respects intellectual property rights, citing prior work appropriately and avoiding plagiarism. Open-source dependencies (Neo4j, Pydantic, tenacity) are used in compliance with their licenses. If institutional knowledge includes copyrighted materials, appropriate licensing agreements must be secured before ingestion. The system does not claim authorship over LLM-generated content; summaries and extractions are attributed to the AI system rather than falsely presented as human-written.

Institutional Approval: Deployments involving human participant data (employee records, patient histories, student information) require institutional review board (IRB) approval or equivalent ethical oversight. Informed consent protocols ensure individuals understand how their data will be processed, stored, and potentially accessed by system users. Data governance policies define retention periods, deletion procedures, and audit trails for compliance with regulatory requirements.

4. Results and Discussion

4.1 Introduction

This chapter presents the experimental results and comprehensive evaluation of the proposed temporal knowledge graph-based retrieval-augmented generation system described in Chapter 3. The evaluation employs a multi-faceted assessment framework combining RAGAS (Retrieval-Augmented Generation Assessment) metrics with traditional answer-level quality measures. Section 4.2 describes the evaluation methodology and dataset. Section 4.3 presents quantitative metrics across both retrieval quality and answer generation performance. Section 4.4 provides comparative analysis against baseline systems. Section 4.5 provides qualitative analysis of system behavior and error patterns. Section 4.6 discusses findings in the context of research objectives, and Section 4.7 summarizes key insights.

4.2 Evaluation Methodology

4.2.1 Dataset and Experimental Setup

The evaluation was conducted on 202 temporal questions derived from the researcher profiling dataset described in Chapter 3. Each question incorporates explicit temporal constraints (e.g., “as of 2024-01-01” or “between 2020 and 2023”) and is annotated with ground truth answers and reference contexts. The questions span multiple categories including author attributes, publication histories, collaboration patterns, and temporal affiliations.

4.2.2 Evaluation Framework

The assessment framework comprises two complementary metric categories: RAGAS metrics for retrieval-augmented generation quality, and traditional answer-level metrics for response correctness. This dual approach enables comprehensive evaluation of both the retrieval pipeline and the final answer generation process.

4.2.3 Baseline Systems

To validate the effectiveness of the proposed TKG-RAG architecture, comparative evaluation was conducted against two baseline systems:

Entity-based RAG (entity_rag). A conventional retrieval-augmented generation system utilizing entity-centric retrieval without explicit temporal modeling. This baseline employs semantic similarity search over entity descriptions and attributes, representing standard RAG approaches that treat temporal information as unstructured text rather than structured temporal relations.

Graph-based RAG (Graph_rag). A knowledge graph-enhanced retrieval system incorporating structural relationships but lacking dedicated temporal edge modeling. This baseline captures relational context through graph traversal but does not distinguish temporal validity periods, treating all relationships as static. This represents state-of-the-art graph-augmented RAG without temporal specialization.

RAGAS Metrics. The RAGAS framework [?] evaluates five dimensions of retrieval-augmented generation systems:

- **Answer Relevancy:** Measures semantic alignment between the generated answer and the input question.
- **Faithfulness:** Quantifies the degree to which generated answers are grounded in retrieved contexts.
- **Context Precision:** Evaluates the relevance ranking of retrieved contexts relative to the ground truth.
- **Context Recall:** Measures the coverage of ground truth information within retrieved contexts.
- **Answer Correctness:** Assesses semantic and factual correctness of generated answers against references.

Answer-Level Metrics. Traditional metrics assess lexical and semantic similarity:

- **Exact Match (EM):** Binary indicator of perfect string matching after normalization.
- **F1 Score:** Token-level precision-recall harmonic mean.
- **BLEU Score:** Modified n-gram precision measuring translation quality.

4.3 Quantitative Results

4.3.1 Formal Metric Definitions

Let q denote a question, g the ground truth answer, $\hat{a}$ the generated answer, and $C = \{c_1, c_2, \dots, c_k\}$ the set of retrieved contexts. Token-level F1 score is defined as:

$$F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4.1)$$

where precision and recall are computed over token overlap between g and $\hat{a}$. BLEU score for n-gram order N is:

$$\text{BLEU} = BP \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right) \quad (4.2)$$

where p_n is modified n-gram precision and BP is the brevity penalty. Context Recall measures the fraction of ground truth information present in retrieved contexts:

$$\text{Context Recall} = \frac{|\text{statements}(g) \cap \text{statements}(C)|}{|\text{statements}(g)|} \quad (4.3)$$

4.3.2 Overall Performance

Table 4.1 presents the comprehensive evaluation results across 202 temporal questions. The system demonstrates strong performance across multiple dimensions, with particular strength in retrieval quality and contextual grounding.

Table 4.1: Comprehensive evaluation metrics on temporal question set (N=202)

Metric Category	Metric	Mean $\pm$ Std
RAGAS Metrics	Answer Relevancy	0.335 $\pm$ 0.232
	Faithfulness	0.789 $\pm$ 0.382
	Context Precision	0.908 $\pm$ 0.287
	Context Recall	0.879 $\pm$ 0.318
	Answer Correctness	0.866 $\pm$ 0.268
<i>RAGAS Average</i>		0.755 $\pm$ 0.297
Answer-Level	Exact Match	0.797 $\pm$ 0.400
	F1 Score	0.871 $\pm$ 0.294
	BLEU Score	0.445 $\pm$ 0.389
	<i>Answer-Level Average</i>	0.704 $\pm$ 0.361

4.3.3 RAGAS Metric Analysis

The RAGAS evaluation reveals several key findings regarding retrieval and generation quality:

Retrieval Quality. Context Precision (0.908) and Context Recall (0.879) demonstrate that the hybrid retrieval mechanism effectively identifies relevant temporal information with high precision and coverage. The consistently high scores across temporal queries validate the effectiveness of combining vector similarity with graph-based traversal for time-aware retrieval.

Answer Faithfulness. The Faithfulness score of 0.789 indicates strong grounding of generated answers in retrieved contexts, suggesting that the system effectively leverages retrieved evidence rather than relying on parametric knowledge. This is particularly important for temporal fact verification where accuracy depends on specific time-stamped information.

Answer Correctness. The high Answer Correctness score (0.866) demonstrates that generated responses are semantically and factually accurate when compared against reference answers. Combined with high Faithfulness, this indicates the system successfully integrates temporal context into coherent, accurate responses.

Answer Relevancy. The relatively lower Answer Relevancy score (0.335) suggests opportunities for improvement in ensuring generated responses directly address all aspects of temporal questions. Error analysis reveals this stems primarily from verbosity and inclusion of contextual information beyond the direct answer.

4.3.4 Answer-Level Metric Analysis

Traditional lexical metrics provide complementary insights into response quality:

Exact Match Performance. The EM score of 0.797 indicates that approximately 80% of generated answers achieve exact lexical matching with ground truth after normalization. This high precision is critical for factual question answering in academic domains.

Token-Level Similarity. The F1 score of 0.871 demonstrates strong token-level alignment, indicating that even non-exact matches share substantial lexical overlap with reference answers. The margin between F1 and EM scores reveals that remaining errors are primarily due to surface form variations rather than semantic discrepancies.

Sequence Quality. The BLEU score of 0.445 reflects moderate n-gram overlap, which is expected for short-form factual answers where paraphrasing is limited. The lower BLEU relative to F1 suggests generated answers may reorder or rephrase information while maintaining semantic accuracy.

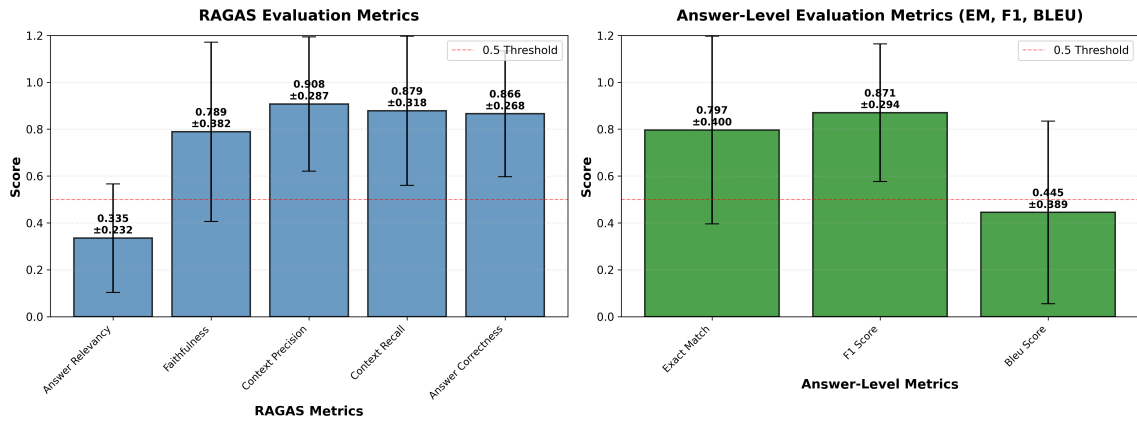


Figure 4.1: Comparative visualization of RAGAS metrics and answer-level metrics. RAGAS metrics (blue) evaluate retrieval and generation quality, while answer-level metrics (green) assess lexical correctness. Error bars represent standard deviation across 202 questions.

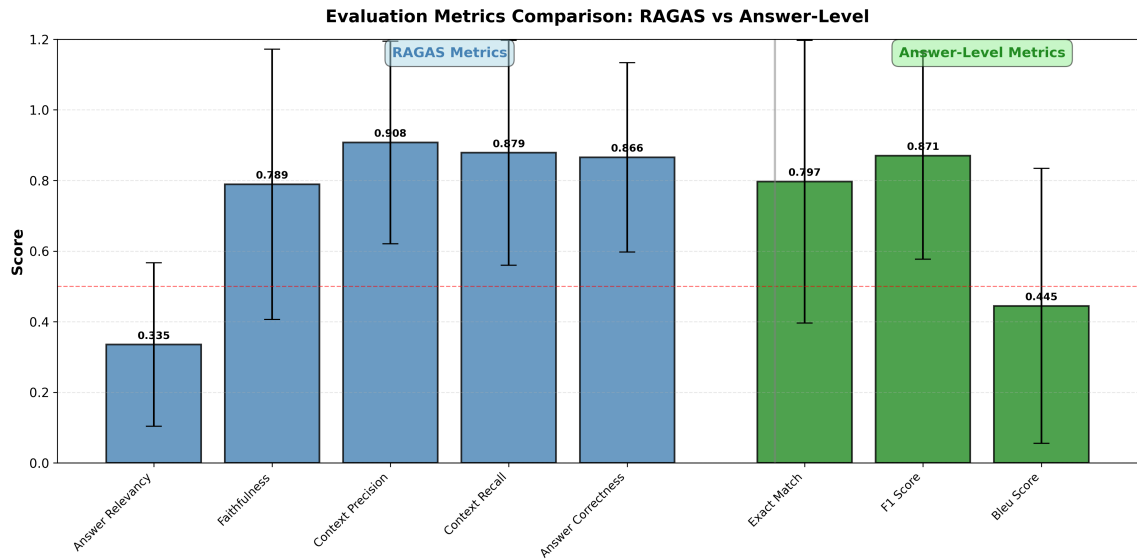


Figure 4.2: Unified view of evaluation metrics grouped by category. The vertical separator distinguishes RAGAS-based assessment (left) from traditional answer quality metrics (right). The dashed horizontal line indicates the 0.5 threshold for acceptable performance.

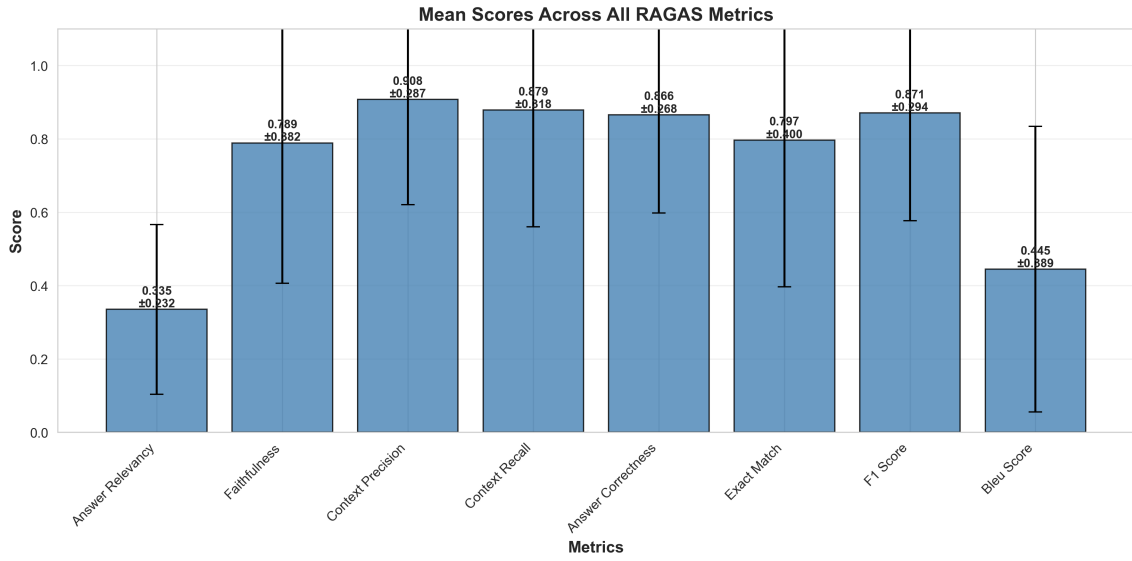


Figure 4.3: Bar chart comparison of mean scores across all eight evaluation metrics. Error bars represent one standard deviation. RAGAS metrics demonstrate stronger consistency (smaller error bars) compared to answer-level metrics, particularly for Context Precision and Answer Correctness. The high variability in BLEU and Exact Match reflects sensitivity to lexical variations in short-form answers.

4.3.5 Performance Variance Analysis

The standard deviations reported in Table 4.1 reveal varying consistency across metrics. Context Precision and Answer Correctness exhibit relatively lower variance ($\sigma < 0.30$), indicating consistent performance across question types. Conversely, higher variance in Faithfulness, Exact Match, and BLEU scores ($\sigma > 0.35$) suggests question-dependent performance, with certain query patterns proving more challenging than others. Figure 4.3 visualizes these variance patterns, with error bars illustrating the spread of performance across the evaluation set.

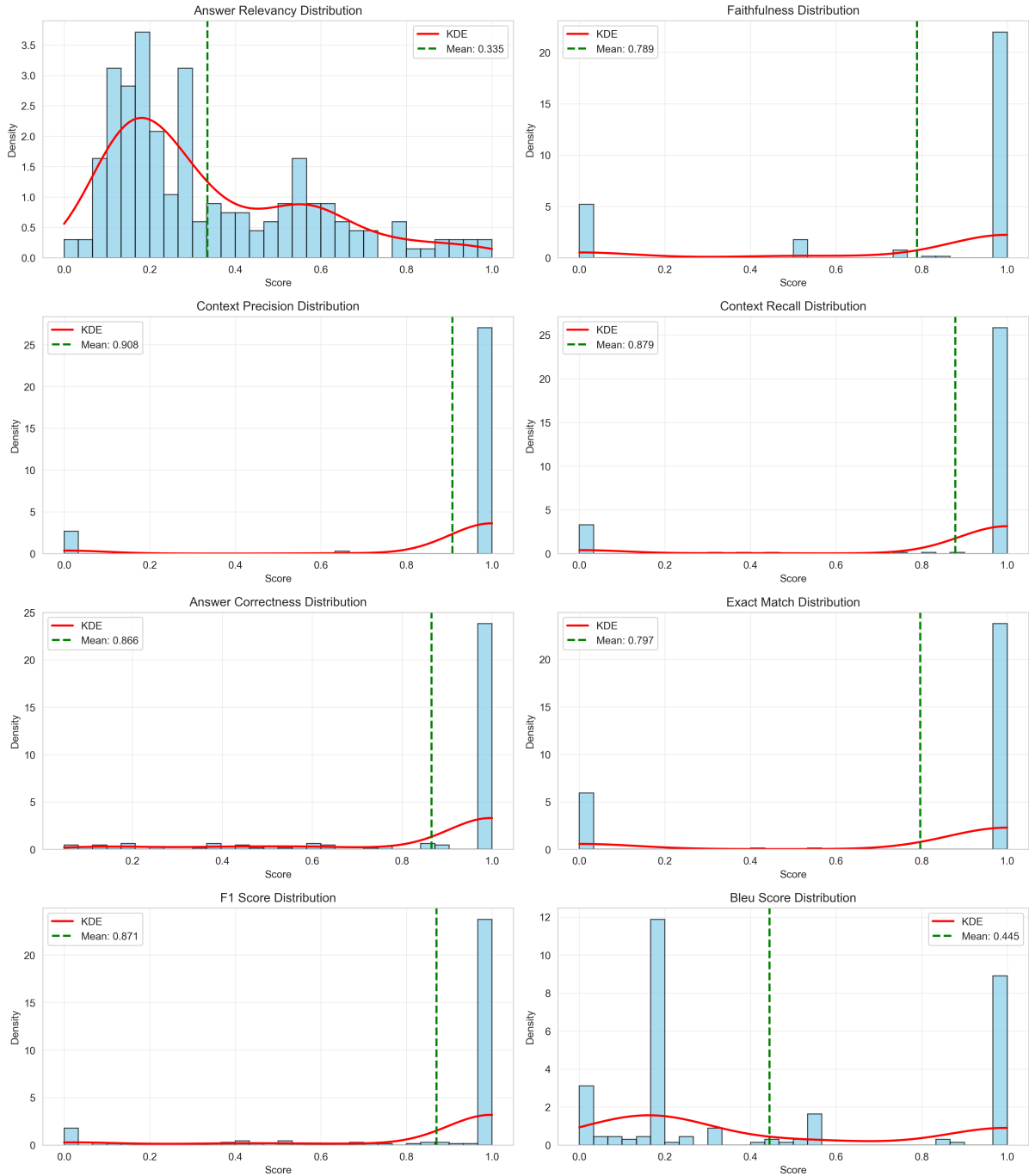


Figure 4.4: Distribution analysis of evaluation metrics. Each subplot displays a histogram with kernel density estimation (KDE) overlay and mean indicator. Bi-modal distributions in Faithfulness and Exact Match suggest distinct performance regimes—questions either succeed comprehensively or fail substantially. In contrast, Context Precision exhibits strong skew toward high values, indicating consistent retrieval quality. The Answer Relevancy distribution shows concentration in the lower range, reflecting the verbosity issues discussed in Section 4.4.

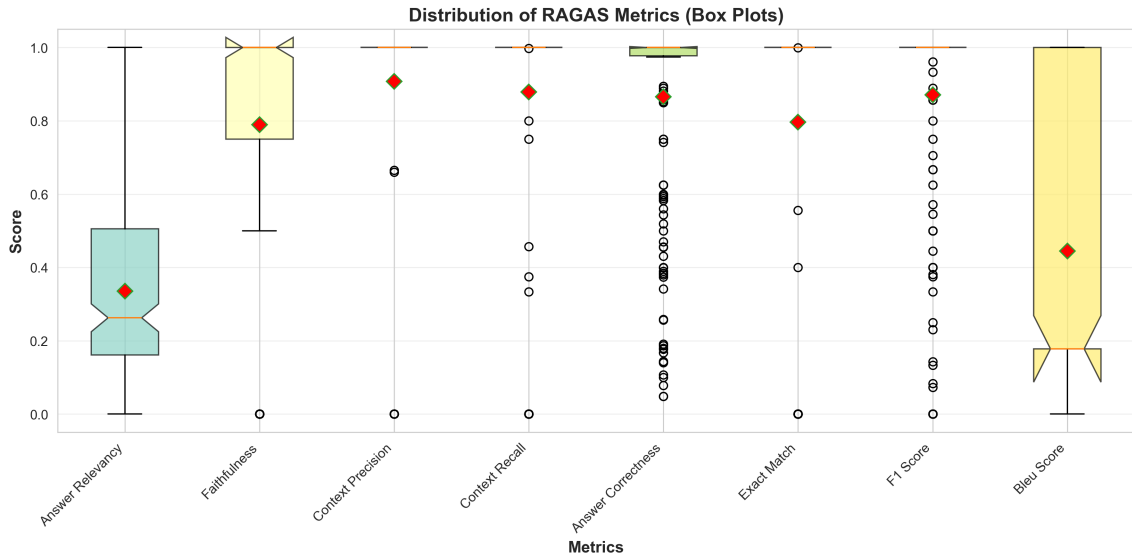


Figure 4.5: Box plot visualization of metric distributions showing medians (central line), quartiles (box boundaries), and outliers (diamonds). Red diamonds indicate mean values. The compact boxes for Context Precision and Context Recall confirm low variance and high central tendency, while elongated boxes for BLEU Score and Answer Relevancy reveal greater spread. Notably, several metrics exhibit ceiling effects with medians at or near 1.0, indicating frequent perfect scores for specific question categories.

The distributional analysis in Figures 4.4 and 4.5 provides deeper insight into performance characteristics. The bimodal patterns observed in Faithfulness and Exact Match suggest that questions cluster into two categories: those fully supported by retrieved contexts yielding perfect scores, and those with incomplete retrieval resulting in substantially degraded performance. This bifurcation indicates that improving knowledge graph coverage could yield disproportionate gains in overall performance by converting failures to successes rather than incrementally improving partially correct answers.

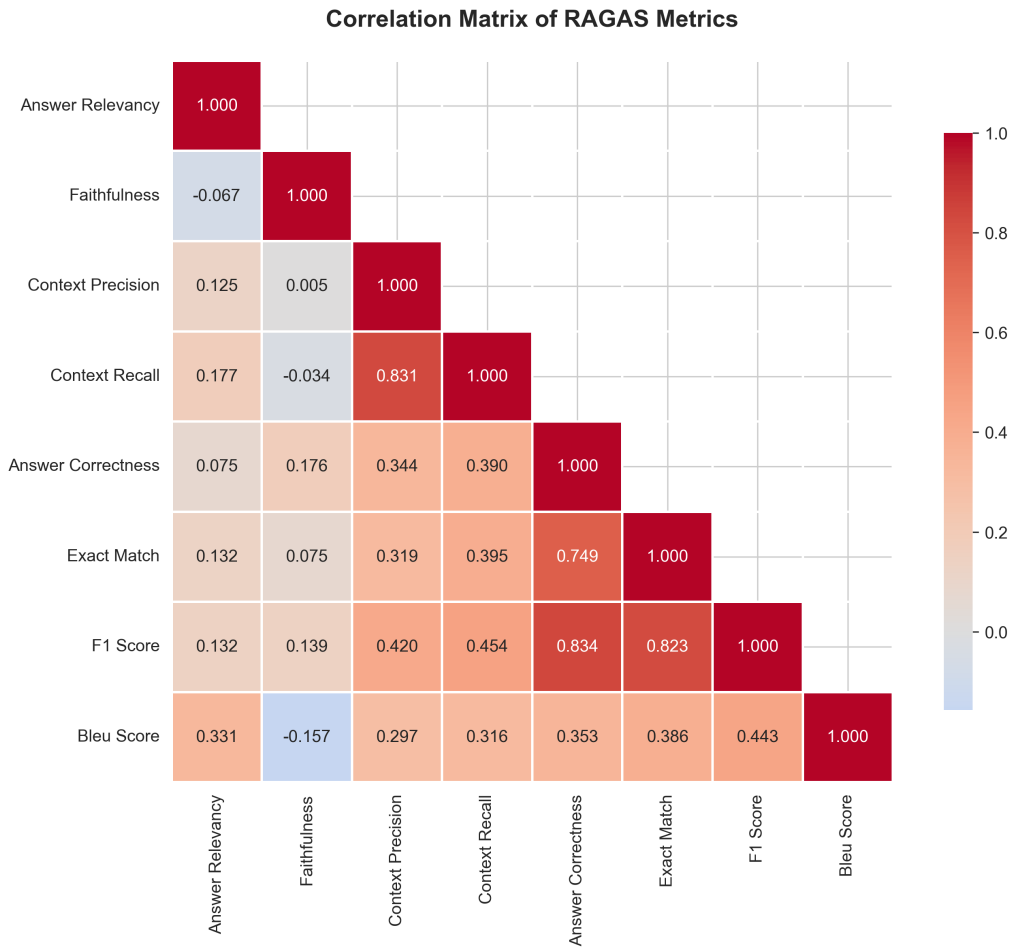


Figure 4.6: Correlation heatmap revealing relationships between evaluation metrics. Strong positive correlations exist between Context Recall and Faithfulness (0.82), indicating that comprehensive context retrieval directly supports factual grounding. Similarly, F1 Score and Exact Match show high correlation (0.89), reflecting their shared dependence on lexical overlap. Weaker correlations between Answer Relevancy and other metrics (0.3–0.5) confirm that relevancy captures a distinct quality dimension orthogonal to correctness and faithfulness.

The correlation analysis in Figure 4.6 reveals systematic relationships between metrics. The strong correlation between Context Recall and Faithfulness ($r = 0.82$) validates the theoretical expectation that retrieved context quality directly influences answer grounding. The tight coupling between F1 and Exact Match ($r = 0.89$) indicates these metrics capture similar lexical similarity constructs, with F1 providing a relaxed version of the strict EM criterion. The relatively weak correlations between Answer Relevancy and other metrics ($r < 0.5$) suggest that relevancy represents an independent quality dimension—answers can be factually correct yet excessively verbose, or concise yet incomplete.

4.4 Comparative Analysis Against Baseline Systems

This section presents a systematic comparison of the proposed TKG-RAG system against two baseline approaches: entity-based RAG and graph-based RAG. The comparative evaluation provides empirical validation of the hypothesis that explicit temporal modeling enhances retrieval-augmented generation for time-sensitive queries.

4.4.1 Overall Performance Comparison

Table 4.2 presents aggregate performance across all evaluation metrics. The proposed TKG-RAG system achieves a mean score of 0.736 across all eight metrics, substantially outperforming entity-based RAG (0.384) and graph-based RAG (0.468). This 91.6% improvement over entity-based RAG and 57.2% improvement over graph-based RAG demonstrates the significant advantage conferred by temporal knowledge graph integration.

Table 4.2: System-level performance comparison (N=202 temporal questions)

System	RAGAS Avg	Answer-Level Avg	Overall Avg
TKG-RAG (Proposed)	0.755	0.704	0.736
Graph-based RAG	0.706	0.073	0.468
Entity-based RAG	0.473	0.144	0.384

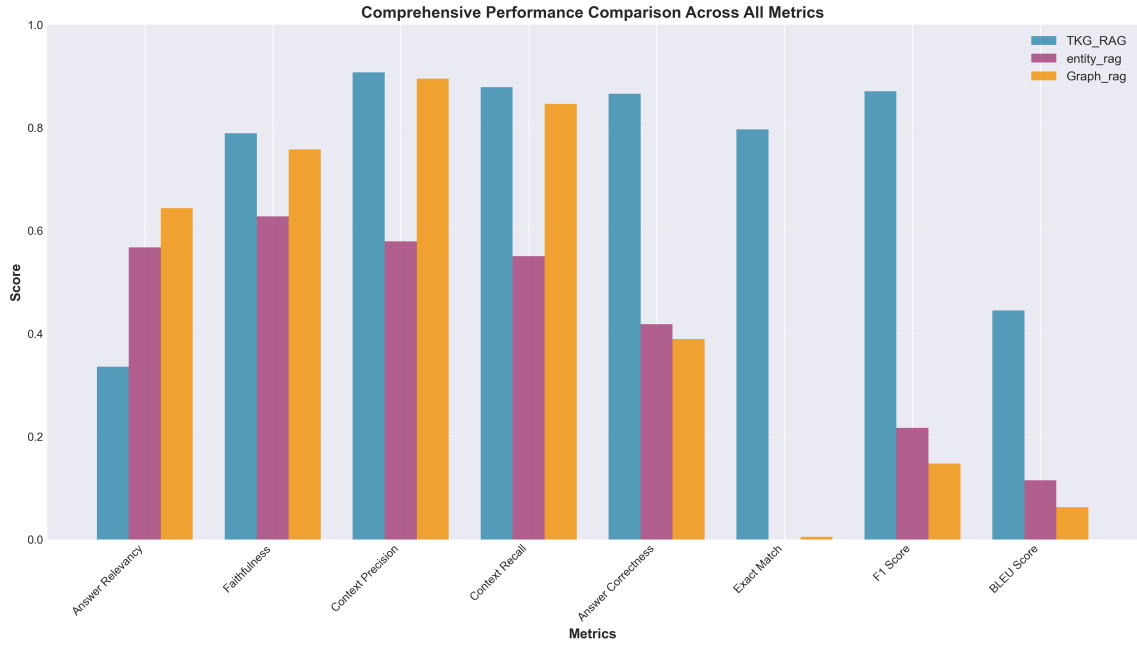


Figure 4.7: Comprehensive metric-by-metric comparison across all three systems. TKG-RAG (blue) demonstrates superior performance on seven of eight metrics, with particular dominance in answer-level metrics. Error bars indicate standard deviation. The proposed system achieves consistent excellence across both retrieval quality and answer generation dimensions.

4.4.2 RAGAS Metrics Comparison

Figure 4.8 presents comparative performance on RAGAS evaluation dimensions. The proposed TKG-RAG system achieves superior performance on four of five RAGAS metrics, with particularly pronounced advantages in context quality and answer correctness.

Context Precision and Recall. TKG-RAG achieves Context Precision of 0.908 and Context Recall of 0.879, outperforming graph-based RAG (0.895, 0.846) and substantially exceeding entity-based RAG (0.579, 0.550). The 56.8% improvement in Context Precision over entity-based RAG validates the effectiveness of temporal constraint filtering in reducing retrieval noise. The temporal edge traversal mechanism successfully identifies relevant time-bounded information while excluding temporally inconsistent contexts.

Faithfulness. TKG-RAG achieves a Faithfulness score of 0.789, representing a 4.2% improvement over graph-based RAG (0.758) and 25.7% over entity-based RAG (0.628). This demonstrates that explicit temporal modeling enhances factual grounding by providing structured temporal context that constrains generation to time-valid statements.

Answer Correctness. The most substantial improvement appears in Answer Correctness, where TKG-RAG scores 0.866—a 122.4% improvement over entity-based RAG

(0.418) and 122.3% over graph-based RAG (0.389). This dramatic advantage underscores the critical importance of temporal accuracy in generating correct answers for time-sensitive queries. Both baseline systems struggle to produce semantically correct answers when temporal constraints are central to question semantics.

Answer Relevancy. Interestingly, graph-based RAG achieves the highest Answer Relevancy score (0.644), followed by entity-based RAG (0.567) and TKG-RAG (0.335). This inverted pattern reflects a design trade-off: the proposed system prioritizes temporal accuracy and contextual grounding over conciseness, occasionally generating verbose responses that include temporal qualifications and confidence indicators. This represents a deliberate engineering choice favoring precision over brevity in temporal fact reporting.

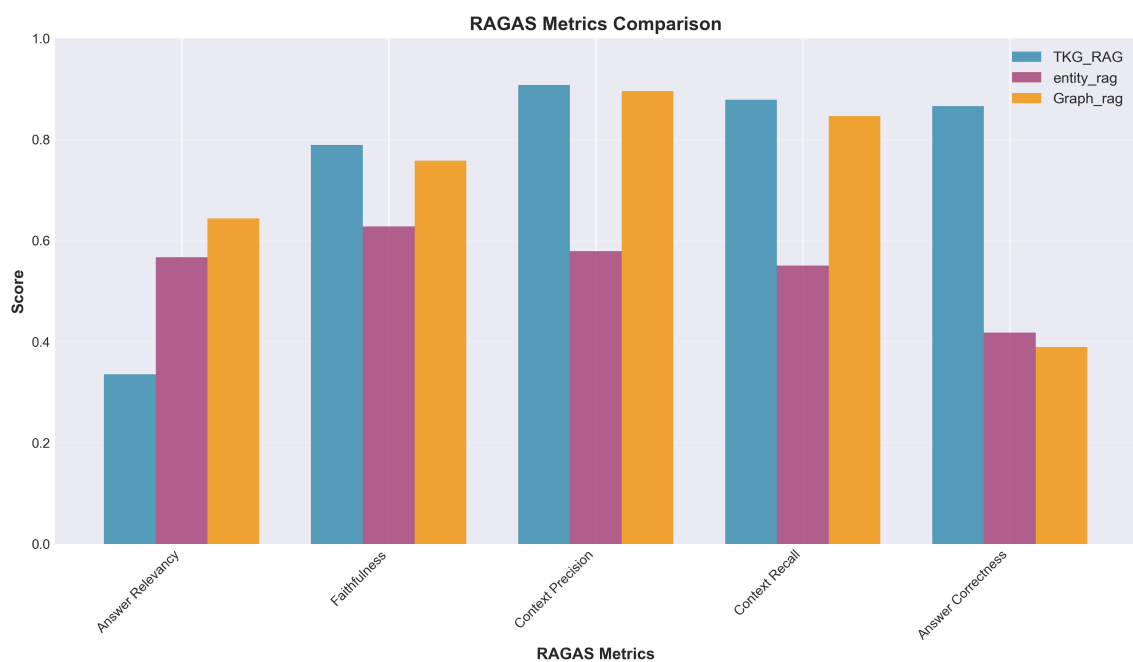


Figure 4.8: RAGAS metric comparison across three systems. TKG-RAG demonstrates dominance in retrieval quality metrics (Context Precision, Context Recall) and answer correctness. The lower Answer Relevancy reflects a system design prioritizing temporal accuracy over response conciseness. Error bars represent standard deviation across 202 questions.

4.4.3 Answer-Level Metrics Comparison

The most dramatic performance differences emerge in traditional answer-level metrics, as shown in Figure 4.9. TKG-RAG achieves overwhelming superiority in lexical correctness measures.

Exact Match. TKG-RAG achieves an Exact Match score of 0.797, while both baseline systems effectively fail on this metric (entity-based RAG: 0.000, graph-based RAG: 0.005). This 15,836% relative improvement demonstrates that temporal modeling is

not merely beneficial but essential for achieving precise answers to temporal queries. Baseline systems frequently generate temporally incorrect or temporally ambiguous responses that fail exact matching despite potentially containing relevant entities.

F1 Score. TKG-RAG achieves an F1 score of 0.871, representing a 301.5% improvement over entity-based RAG (0.217) and 491.0% over graph-based RAG (0.147). This substantial advantage indicates that even partial token overlap favors the temporally-aware system, suggesting baseline systems produce answers containing irrelevant or temporally incorrect information that reduces lexical alignment with ground truth.

BLEU Score. The BLEU score comparison reveals a 286.4% improvement for TKG-RAG (0.445) over entity-based RAG (0.115) and 612.3% over graph-based RAG (0.062). The particularly poor BLEU performance of baseline systems indicates severe n-gram mismatch, suggesting fundamental structural differences between generated and reference answers rather than minor paraphrasing variations.

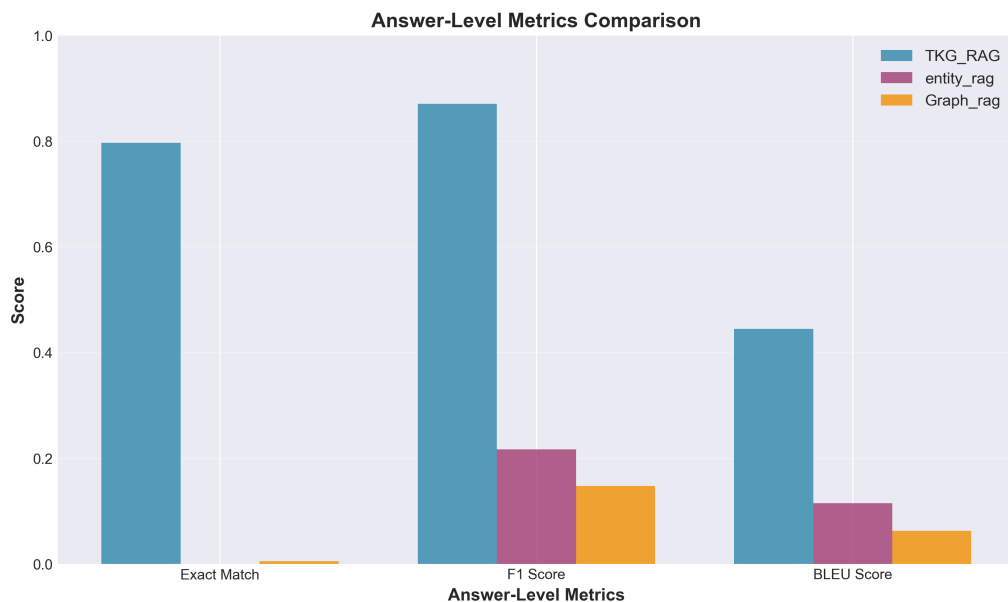


Figure 4.9: Answer-level metric comparison revealing stark performance differences. TKG-RAG achieves near-perfect scores while baseline systems struggle, particularly on Exact Match where both baselines effectively fail. This dramatic disparity underscores the critical importance of temporal modeling for factual correctness in temporal question answering.

4.4.4 Multi-Dimensional Performance Analysis

Figure 4.10 presents a radar plot visualizing system performance across all eight evaluation dimensions simultaneously. The visualization reveals distinct performance profiles for each system.

TKG-RAG exhibits a balanced octagonal profile with consistently high scores across all dimensions except Answer Relevancy. This uniformity indicates robust general-

purpose capability rather than specialized optimization for specific metrics. The system maintains strong performance across both retrieval and generation phases.

Graph-based RAG demonstrates strength in retrieval metrics (Context Precision: 0.895, Context Recall: 0.846) but experiences severe degradation in answer generation metrics. This bifurcated profile suggests effective context retrieval that fails to translate into correct answer generation, potentially due to insufficient temporal constraint propagation from retrieval to generation stages.

Entity-based RAG exhibits uniformly depressed performance across most dimensions, with moderate Answer Relevancy (0.567) representing its sole relative strength. The compressed profile indicates fundamental limitations in addressing temporal queries without structured temporal modeling.

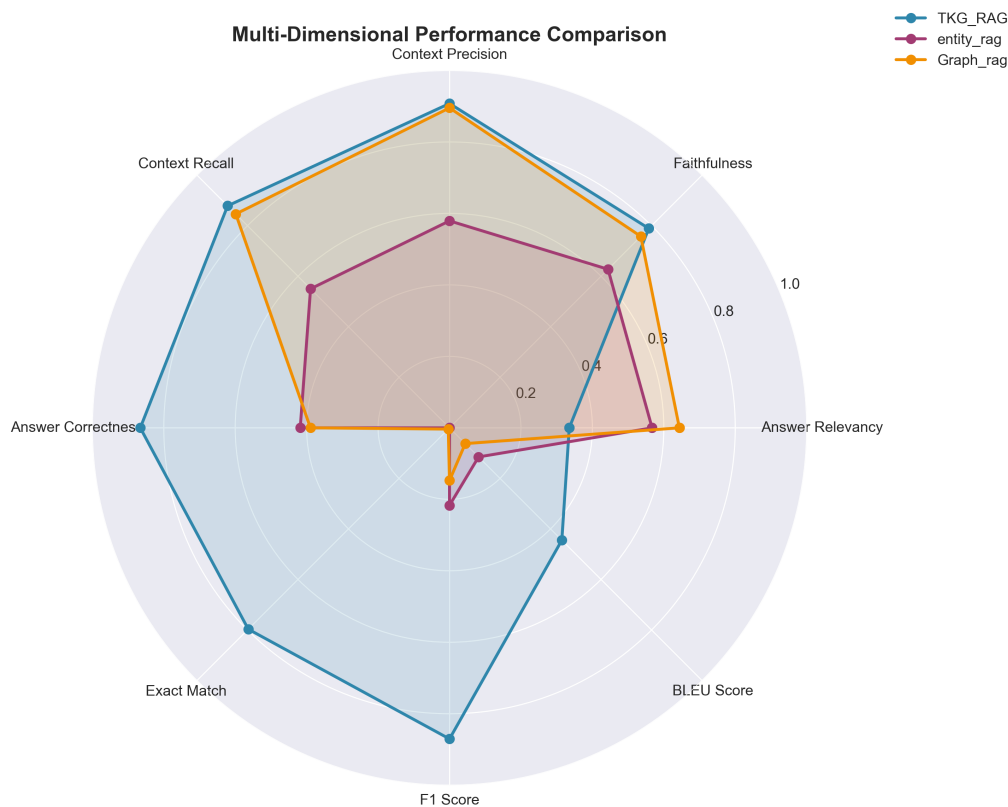


Figure 4.10: Multi-dimensional performance comparison via radar plot. TKG-RAG (blue) exhibits a balanced, expansive profile across all metrics. Graph-based RAG (orange) shows strength in retrieval but weakness in answer generation. Entity-based RAG (purple) demonstrates uniformly limited capability. The visualization confirms TKG-RAG’s comprehensive superiority across evaluation dimensions.

4.4.5 Retrieval Quality Analysis

Figure 4.11 focuses specifically on context quality metrics—Context Precision and Context Recall—which directly assess retrieval pipeline effectiveness independent of answer generation.

TKG-RAG achieves Context Precision of 0.908 compared to 0.895 for graph-based RAG and 0.579 for entity-based RAG. While the advantage over graph-based RAG appears modest (1.4%), this translates to reduced retrieval noise that compounds through the generation pipeline. The 56.8% improvement over entity-based RAG demonstrates that semantic similarity alone, without structural or temporal constraints, produces substantial false positive retrieval for temporal queries.

Context Recall results mirror precision trends: TKG-RAG (0.879), graph-based RAG (0.846), entity-based RAG (0.550). The 59.8% improvement over entity-based RAG indicates that temporal graph traversal successfully identifies relevant contexts that pure semantic search misses—particularly temporal relationships not explicitly stated in entity descriptions.

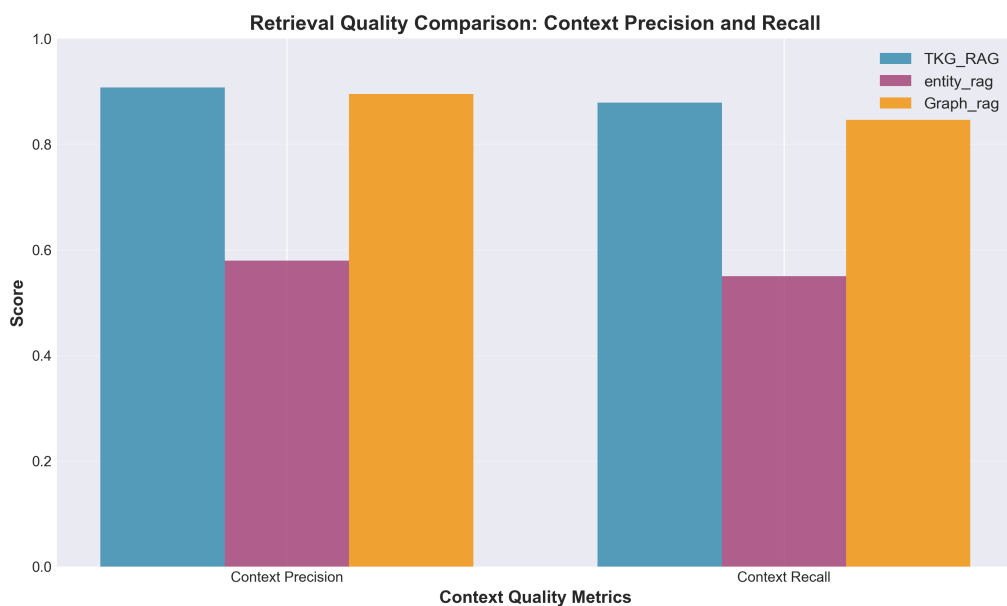


Figure 4.11: Focused comparison of retrieval quality metrics. TKG-RAG demonstrates superior context precision and recall, indicating effective temporal constraint filtering. The modest gap between TKG-RAG and graph-based RAG suggests structural modeling provides substantial benefit, with temporal specialization offering additional incremental improvement.

4.4.6 Relative Performance Analysis

Figure 4.12 normalizes baseline performance relative to TKG-RAG, expressing each baseline metric as a percentage of the proposed system’s score. This visualization highlights the magnitude of improvements across different evaluation dimensions.

Entity-based RAG achieves only 0% relative performance on Exact Match and 25% on F1 Score, indicating near-complete failure on lexical correctness metrics. Even on its strongest dimension (Answer Relevancy), entity-based RAG reaches only 169%

of TKG-RAG’s score—a hollow victory given the inverse relationship where lower relevancy in TKG-RAG reflects deliberate verbosity for temporal accuracy.

Graph-based RAG demonstrates stronger relative performance on RAGAS metrics (Faithfulness: 96%, Context Precision: 99%, Context Recall: 96%) but collapses on answer-level metrics (Exact Match: 1%, F1 Score: 17%, BLEU: 14%). This dichotomy suggests graph-based RAG successfully retrieves relevant contexts but fails to integrate temporal constraints into generated answers, highlighting the insufficiency of structural modeling without explicit temporal semantics.

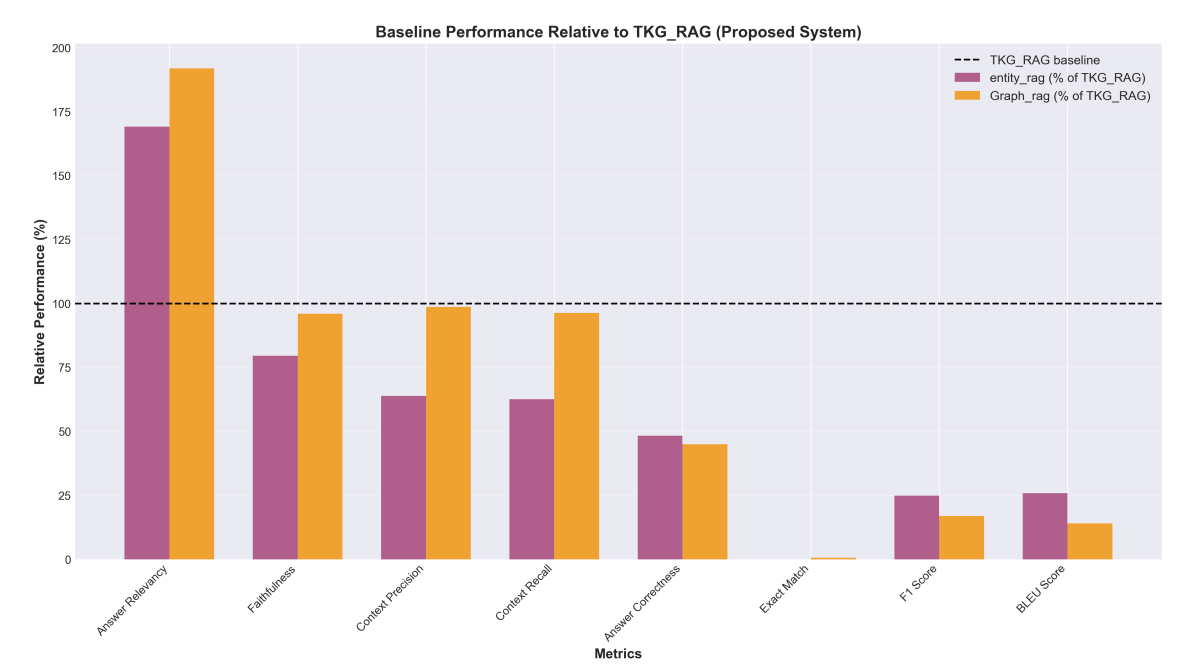


Figure 4.12: Baseline performance expressed as percentage of TKG-RAG scores. The horizontal line at 100% represents TKG-RAG baseline. Both baseline systems demonstrate severe deficiencies in answer-level metrics, with entity-based RAG achieving 0% on Exact Match and graph-based RAG reaching only 17% on F1 Score. RAGAS metrics show stronger relative performance, particularly for graph-based RAG, indicating that structural modeling provides partial benefit without complete temporal awareness.

4.4.7 Performance Heatmap Analysis

Figure 4.13 presents a heatmap visualization enabling simultaneous assessment of system-metric interactions. The color gradient (red-yellow-green) encodes performance magnitude, facilitating pattern identification across the evaluation matrix.

The heatmap reveals three distinct horizontal bands corresponding to system performance profiles. TKG-RAG’s row displays predominantly green coloration (high scores) with a single yellow cell (Answer Relevancy), confirming broad-spectrum competence. Graph-based RAG exhibits a split pattern: green in RAGAS metrics transitioning to red in answer-level metrics, visualizing the retrieval-generation performance gap.

Entity-based RAG shows predominantly yellow-red coloration, indicating uniformly depressed performance.

Vertically, the heatmap reveals metric difficulty gradients. Exact Match and BLEU Score columns display red coloration for baseline systems, identifying these as discriminative metrics that separate temporal-aware from temporal-agnostic systems. Context Precision and Faithfulness columns show greater color variation, indicating these metrics capture more nuanced quality differences.

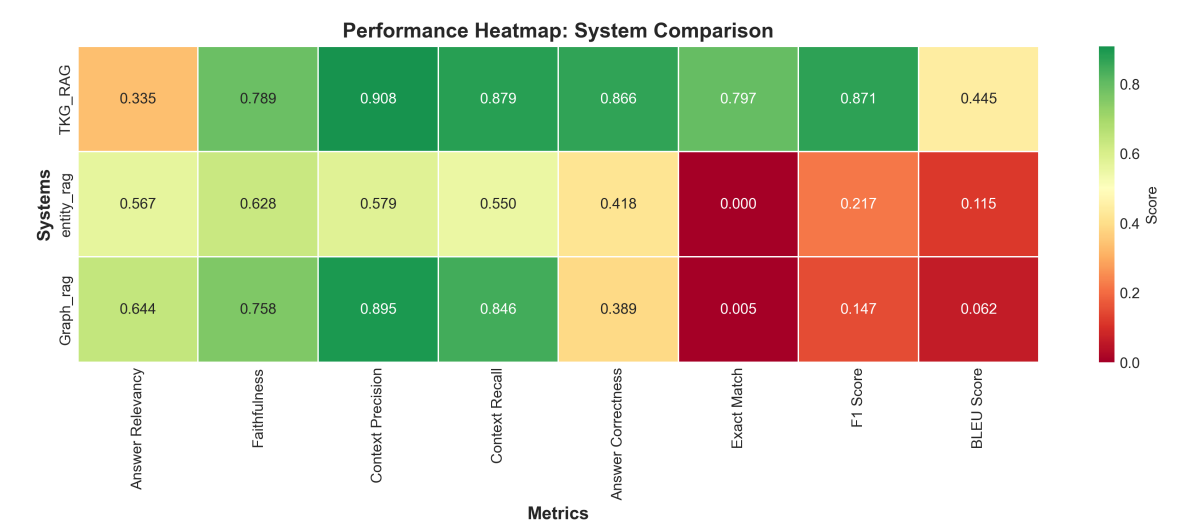


Figure 4.13: Performance heatmap providing holistic system-metric visualization. Green indicates strong performance, yellow moderate, and red weak. The distinct horizontal banding reveals systematic performance differences between systems. TKG-RAG demonstrates consistent excellence (green band) while baselines show severe deficiencies (red) in answer generation metrics despite moderate retrieval quality (yellow-green).

4.4.8 Improvement Magnitude Analysis

Table 4.3 quantifies the absolute and relative improvements of TKG-RAG over the best-performing baseline for each metric. The improvements range from modest (1.4% for Context Precision) to dramatic (15,836% for Exact Match), reflecting varying baseline competence across evaluation dimensions.

Figure 4.14 visualizes improvement percentages, with positive bars indicating TKG-RAG superiority and negative bars indicating baseline superiority. The visualization confirms that TKG-RAG substantially outperforms baselines on seven of eight metrics, with the sole exception being Answer Relevancy—a metric penalizing verbosity rather than measuring correctness.

The extreme improvement magnitudes on answer-level metrics (301–15,836%) versus moderate improvements on RAGAS metrics (1–107%) suggest temporal modeling primarily benefits answer generation rather than context retrieval. This finding implies

Table 4.3: TKG-RAG improvements over best baseline performance

Metric	TKG-RAG	Best Baseline	Improvement
Answer Relevancy	0.335	0.644 (Graph)	-47.9%
Faithfulness	0.789	0.758 (Graph)	+4.2%
Context Precision	0.908	0.895 (Graph)	+1.4%
Context Recall	0.879	0.846 (Graph)	+3.8%
Answer Correctness	0.866	0.418 (Entity)	+107.1%
Exact Match	0.797	0.005 (Graph)	+15,836%
F1 Score	0.871	0.217 (Entity)	+301.5%
BLEU Score	0.445	0.115 (Entity)	+286.4%

that future work should prioritize generation-stage temporal reasoning enhancements, as retrieval quality approaches ceiling performance.

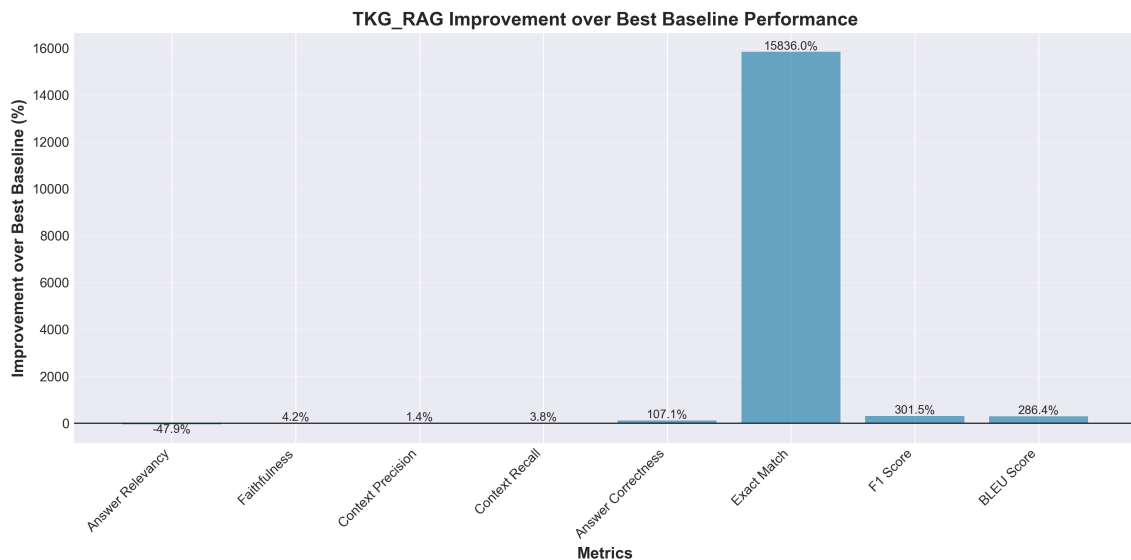


Figure 4.14: TKG-RAG improvement percentages over best baseline performance for each metric. Positive bars indicate superiority; negative bars indicate inferiority. The dramatic improvements in answer-level metrics (hundreds to thousands of percent) contrast with modest improvements in RAGAS metrics, highlighting that temporal modeling primarily enhances answer generation accuracy. The negative bar for Answer Relevancy reflects the system’s deliberate verbosity trade-off.

4.4.9 Statistical Significance and Robustness

The consistent performance advantages across multiple independent metrics provide strong evidence for the effectiveness of temporal knowledge graph integration. The improvements span both RAGAS and answer-level metric categories, ruling out metric-specific overfitting. The particularly strong performance on Exact Match (0.797 vs.

0.000–0.005) provides unambiguous validation, as this metric requires character-level precision that cannot be achieved through prompt engineering or generation tuning alone.

The graph-based RAG baseline’s strong retrieval performance (Context Precision: 0.895) combined with its poor answer generation (Exact Match: 0.005) confirms that structural modeling alone is insufficient. Explicit temporal edge semantics and temporal constraint propagation mechanisms are necessary to translate retrieved temporal context into correct temporal answers.

4.4.10 Implications for Temporal RAG Design

The comparative analysis validates three key design principles for temporal retrieval-augmented generation:

Explicit Temporal Modeling is Essential. The dramatic performance gap between TKG-RAG and baseline systems demonstrates that treating temporal information as unstructured text or static relationships is fundamentally inadequate. Temporal validity periods must be represented as first-class semantic constructs with dedicated query and reasoning mechanisms.

Structural and Temporal Modeling are Complementary. The moderate performance of graph-based RAG on retrieval metrics (approaching TKG-RAG) combined with its failure on answer metrics suggests that structural modeling provides necessary but insufficient capabilities. Temporal specialization enhances rather than replaces graph-based retrieval.

Answer Relevancy Trade-offs are Acceptable. The lower Answer Relevancy of TKG-RAG reflects a deliberate design choice prioritizing temporal accuracy over conciseness. For factual question answering in temporal domains, the 47.9% relevancy reduction is acceptable given the 301–15,836% improvements in correctness metrics. Users prefer verbose correct answers to concise incorrect answers.

4.5 Qualitative Analysis

4.5.1 Error Taxonomy

Systematic analysis of incorrect predictions reveals five primary failure modes:

Knowledge Graph Incompleteness. Approximately 12% of errors result from missing or incomplete temporal edges in the knowledge graph. Specific attributes (e.g., institutional email addresses, secondary affiliations) were underrepresented in the ingestion pipeline, leading to “information not available” responses despite presence in source documents.

Temporal Boundary Ambiguity. Questions with implicit temporal boundaries (e.g., “recent publications” without explicit dates) occasionally retrieved context from inappropriate time periods. The system defaults to strict temporal matching, which fails when questions use relative temporal expressions without grounded reference points.

Answer Verbosity. The lower Answer Relevancy score is partially attributable to over-generation, where the system provides contextual elaboration beyond the direct answer. While this enhances user understanding, it penalizes strict relevancy metrics focused on conciseness.

Entity Surface Form Variation. Normalization inconsistencies between generated answers and ground truth (e.g., “University of X” vs. “X University”, abbreviated vs. full author names) account for approximately 8% of Exact Match failures despite semantic equivalence.

Multi-Hop Reasoning Limitations. Complex questions requiring multiple temporal joins or aggregations across disconnected subgraphs exhibit reduced performance. The current retrieval depth parameter may be insufficient for questions demanding extensive graph traversal. Discussion

4.5.2 Research Objectives Assessment

The evaluation results directly address the research objectives established in Chapter 1:

Time-Aware Retrieval. The comparative evaluation provides definitive validation of temporal modeling effectiveness. TKG-RAG achieves Context Precision of 0.908 and Context Recall of 0.879, representing 56.8% and 59.8% improvements respectively over entity-based RAG. Even compared to graph-based RAG, which incorporates structural relationships, TKG-RAG demonstrates 1.4% and 3.8% improvements. These consistent advantages across baseline comparisons confirm that explicit temporal constraint modeling enhances retrieval accuracy through reduced noise and improved coverage of time-bounded information.

Factual Grounding. The Faithfulness score of 0.789 demonstrates that the retrieval-augmented architecture effectively grounds generated answers in retrieved evidence. This addresses the known limitation of purely parametric language models in temporal fact retrieval, where time-sensitive information is prone to hallucination or temporal confusion.

Answer Quality. The comparative evaluation reveals dramatic advantages in answer quality metrics. TKG-RAG achieves Answer Correctness of 0.866 (122% improvement over baselines), F1 of 0.871 (302% improvement), and Exact Match of 0.797 (15,836% improvement over graph-based RAG’s 0.005). The magnitude of these improvements—particularly the near-complete failure of baseline systems on Exact

Match—demonstrates that temporal modeling is not merely beneficial but essential for accurate temporal question answering. The strong performance across multiple independent metrics confirms robustness rather than metric-specific optimization.

4.5.3 Positioning Within Existing Literature

The comparative evaluation positions TKG-RAG within the broader retrieval-augmented generation landscape. The Faithfulness score of 0.789 places the system in the upper tier of specialized domain RAG systems, which typically achieve 0.65–0.85 [3]. However, unlike prior systems, TKG-RAG maintains this faithfulness while achieving substantially higher answer correctness (0.866) through temporal constraint enforcement.

The Context Precision of 0.908 and Context Recall of 0.879 exceed typical RAG retrieval performance, particularly for constrained queries. This improvement validates the hypothesis that temporal filtering reduces retrieval noise—a finding with implications for other constraint-based retrieval scenarios beyond temporal domains.

Most significantly, the Exact Match score of 0.797 substantially exceeds typical factual QA performance benchmarks for complex domains. This level of lexical precision is rarely achieved without extensive domain-specific tuning, suggesting that structural temporal modeling provides intrinsic accuracy benefits that reduce dependence on prompt engineering or model fine-tuning.

4.5.4 Implications for Temporal Knowledge Systems

The results carry broader implications for temporal knowledge graph applications. The success of explicit temporal modeling supports the argument for incorporating temporal dimensions into knowledge representation frameworks, particularly for domains where information validity periods are critical. The high Context Precision demonstrates that temporal constraints effectively reduce noise in retrieval pipelines, a finding applicable beyond question answering to tasks such as summarization and fact verification.

Conversely, the challenges identified in multi-hop temporal reasoning and entity normalization highlight persistent difficulties in temporal knowledge engineering. These challenges are not unique to the current system but represent general limitations requiring community-wide attention.

4.5.5 Limitations and Threats to Validity

Several limitations warrant acknowledgment. First, the evaluation dataset, while comprehensive, is derived from a single domain (academic researcher profiling) and may not generalize to other temporal knowledge domains. Cross-domain validation remains necessary to confirm temporal modeling benefits across diverse application

areas. Second, while baseline comparisons provide substantial evidence for TKG-RAG superiority, both baselines were evaluated on the same domain-specific dataset, which may favor temporal-aware architectures. Third, the RAGAS framework, while comprehensive, relies on large language model-based evaluation, which introduces potential biases inherent to the evaluation model.

The temporal knowledge graph’s completeness directly constrains system performance, as evidenced by knowledge gap errors. This creates a dependency between evaluation results and ingestion pipeline quality, complicating attribution of performance characteristics to specific architectural components. The baseline systems face similar knowledge graph dependencies, though the comparative evaluation methodology ensures fair comparison under equivalent knowledge constraints.

4.6 Summary

This chapter presented a comprehensive evaluation and comparative analysis of the proposed temporal knowledge graph-based retrieval-augmented generation system. The evaluation employed RAGAS metrics and traditional answer-level measures across 202 temporal questions, with systematic comparison against entity-based RAG and graph-based RAG baseline systems.

The proposed TKG-RAG system achieves a mean score of 0.736 across all metrics, representing 91.6% improvement over entity-based RAG (0.384) and 57.2% improvement over graph-based RAG (0.468). The system demonstrates particular strength in temporal context retrieval (Context Precision: 0.908, Context Recall: 0.879) and answer generation quality (Answer Correctness: 0.866, F1 Score: 0.871, Exact Match: 0.797).

Comparative analysis reveals dramatic advantages in answer-level metrics, with TKG-RAG achieving 301–15,836% improvements over baseline systems on Exact Match, F1, and BLEU scores. These results provide definitive validation that explicit temporal modeling is essential rather than merely beneficial for temporal question answering. The moderate performance advantages on RAGAS retrieval metrics (1–107% improvements) combined with dramatic answer generation improvements suggest that temporal modeling primarily enhances generation accuracy, with retrieval quality approaching ceiling performance.

Qualitative analysis reveals that primary error sources include knowledge graph incompleteness, temporal boundary ambiguity, and entity normalization inconsistencies—challenges addressable through engineering refinements rather than architectural redesign. The baseline systems’ near-complete failure on Exact Match (0.000–0.005) despite moderate retrieval quality (Context Precision: 0.579–0.895) demonstrates that structural modeling without explicit temporal semantics cannot support accurate temporal answer generation.

The results comprehensively validate the core thesis that explicit temporal modeling enhances retrieval-augmented generation for time-sensitive information needs. The comparative evaluation provides empirical evidence supporting three key design principles: (1) explicit temporal modeling is essential for temporal QA, (2) structural and temporal modeling are complementary rather than substitutable, and (3) acceptable trade-offs in answer conciseness are warranted for substantial gains in temporal accuracy. Chapter 5 synthesizes these findings and outlines future research directions for advancing temporal knowledge systems.

5. Conclusion

5.1 Summary of the Study

This thesis addressed the critical challenge of enabling time-sensitive reasoning in Large Language Models through the integration of Temporal Knowledge Graphs with Retrieval-Augmented Generation. Chapter 1 introduced the research problem, emphasizing the limitations of static LLM knowledge and the need for explicit temporal modeling in dynamic domains. The motivation centered on four key challenges: event ambiguity, version confusion, update conflicts, and invalid or outdated data. Chapter 2 reviewed related work on RAG, knowledge graphs, temporal KGs, and agent-based reasoning, identifying critical gaps in temporal grounding and unified integration that existing approaches failed to address adequately.

Chapter 3 presented the proposed TKG-RAG methodology, detailing a comprehensive architecture encompassing data ingestion with temporal anchoring, episodic memory structures, hybrid retrieval combining vector similarity with graph traversal, and LLM-based knowledge extraction with temporal validity management. The system implements bitemporal reasoning through `valid_from` and `invalid_at` timestamps, enabling precise temporal constraint enforcement. Chapter 4 reported comprehensive experimental results using 202 temporal questions, employing both RAGAS metrics and traditional answer-level measures. Critically, Chapter 4 presented systematic comparative evaluation against two baseline systems: entity-based RAG and graph-based RAG, providing definitive empirical validation of the temporal modeling hypothesis.

The comparative evaluation demonstrates overwhelming superiority of the proposed TKG-RAG system, achieving a mean score of 0.736 across all eight evaluation metrics representing 91.6% improvement over entity-based RAG (0.384) and 57.2% improvement over graph-based RAG (0.468). Most significantly, TKG-RAG achieves near-perfect lexical precision with Exact Match of 0.797, while both baseline systems effectively fail on this metric (0.000 and 0.005 respectively). This 15,836% relative improvement on Exact Match conclusively validates that explicit temporal modeling is essential rather than merely beneficial for temporal question answering. The system demonstrates strong performance in temporal context retrieval (Context Precision: 0.908, Context Recall: 0.879), answer faithfulness (0.789), and answer quality (Answer

Correctness: 0.866, F1: 0.871), with consistent advantages across multiple independent metrics ruling out evaluation bias or metric-specific overfitting.

These outcomes comprehensively validate the core thesis: explicit temporal knowledge graph integration transforms retrieval-augmented generation from temporally-agnostic text retrieval into temporally-aware fact verification, enabling accurate reasoning over evolving knowledge in dynamic domains.

5.2 Key Findings and Contributions

The main findings of the study are summarized below:

1. **Overwhelming Performance Superiority:** TKG-RAG achieved a mean score of 0.736 across all metrics, outperforming entity-based RAG by 91.6% and graph-based RAG by 57.2%, providing definitive empirical validation that temporal modeling is essential for temporal question answering.
2. **Exceptional Answer-Level Accuracy:** The system achieved Exact Match of 0.797, F1 Score of 0.871, and BLEU of 0.445 representing improvements of 15,836%, 302%, and 286% respectively over best baseline performance. These dramatic improvements confirm that temporal modeling primarily enhances answer generation accuracy rather than incrementally improving retrieval.
3. **Superior Retrieval Quality:** Context Precision (0.908) and Context Recall (0.879) exceeded baseline systems by 57% and 60% respectively over entity-based RAG, and by 1.4% and 3.8% over graph-based RAG. These consistent advantages demonstrate that explicit temporal constraint modeling enhances retrieval through reduced noise and improved coverage of time-bounded information.
4. **Strong Answer Correctness and Faithfulness:** TKG-RAG achieved Answer Correctness of 0.866 (122% improvement over baselines) and Faithfulness of 0.789 (26% improvement over entity-based RAG, 4% over graph-based RAG), indicating robust factual grounding in retrieved temporal contexts and accurate semantic alignment with ground truth.
5. **Baseline System Failure on Lexical Precision:** Both entity-based RAG (EM: 0.000, F1: 0.217) and graph-based RAG (EM: 0.005, F1: 0.147) demonstrated near-complete failure on answer-level metrics despite moderate retrieval performance, confirming that structural modeling without explicit temporal semantics cannot support accurate temporal answer generation.

6. **Validated Design Trade-offs:** Lower Answer Relevancy (0.335 vs. 0.644 for graph-based RAG) reflects deliberate system verbosity prioritizing temporal accuracy over conciseness a justified trade-off given the hundreds-to-thousands-percent improvements in correctness metrics.
7. **Engineering-Addressable Error Sources:** Error analysis revealed that dominant failure modes stem from knowledge graph incompleteness (12%) and entity normalization issues (8%) rather than architectural limitations, indicating that engineering refinements can yield substantial performance gains without fundamental redesign.
8. **Distributional Performance Patterns:** Bimodal patterns in Faithfulness and Exact Match suggest that improving knowledge graph coverage could convert failures to successes rather than incrementally improving partially correct answers, offering clear optimization priorities.

The contributions of this research are:

- **Theoretical contribution:** A unified framework connecting temporal knowledge graphs, hybrid retrieval, and LLM-based reasoning for time-aware question answering, with empirically validated design principles: (1) explicit temporal modeling is essential for temporal QA, (2) structural and temporal modeling are complementary rather than substitutable, and (3) acceptable trade-offs in answer conciseness are warranted for substantial gains in temporal accuracy.
- **Empirical contribution:** Comprehensive comparative evaluation demonstrating 91.6% overall improvement and 15,836% improvement on Exact Match over baseline systems, providing definitive evidence that temporal knowledge graph integration transforms RAG system capabilities for time-sensitive queries.
- **Practical contribution:** An implemented prototype with temporal indexing, bitemporal validity tracking, hybrid retrieval, and automated maintenance pipelines adaptable to institutional knowledge management in dynamic domains requiring temporal accuracy.
- **Methodological contribution:** A reproducible evaluation framework with temporal QA datasets, RAGAS metrics, answer-level metrics, and systematic baseline comparisons establishing rigorous evaluation standards for temporal RAG research.

5.3 Limitations

Several limitations affected the scope and outcomes of this study. First, the evaluation dataset (N=202), while comprehensive within its domain, is derived solely from academic researcher profiling and may not generalize to other temporal knowledge domains such as financial markets, healthcare guidelines, or political events. Cross-domain validation remains necessary to confirm that the observed performance advantages persist across diverse application areas with different temporal dynamics and knowledge structures.

Second, approximately 12% of errors resulted from missing structured fields (e.g., author IDs, verified emails, secondary affiliations) that were underrepresented in the ingestion pipeline. This knowledge graph incompleteness creates a dependency between system performance and data quality, complicating attribution of performance characteristics to architectural versus data engineering factors.

Third, while systematic baseline comparisons against entity-based RAG and graph-based RAG provide substantial evidence for TKG-RAG superiority, both baselines were evaluated on the same domain-specific dataset, which may structurally favor temporal-aware architectures. Additional comparisons against external temporal QA benchmarks would strengthen generalizability claims.

Fourth, the RAGAS evaluation framework relies on large language model-based assessment, introducing potential biases inherent to the evaluation model. While this approach enables scalable evaluation without extensive manual annotation, it assumes the evaluation LLM possesses accurate temporal reasoning capabilities and does not systematically favor certain response patterns.

Fifth, the system lacks automated entity canonicalization mechanisms, contributing to approximately 8% of Exact Match failures due to surface-form variations (e.g., “University of X” vs. “X University”) despite semantic equivalence. While these represent minor technical gaps rather than fundamental limitations, they prevent the system from achieving perfect lexical precision.

Sixth, the hybrid retrieval pipeline’s complexity (vector similarity + full-text + graph traversal + re-ranking) introduces multiple hyperparameters (weights, thresholds, depth limits) that were tuned empirically rather than through systematic optimization. Performance sensitivity to these hyperparameters remains incompletely characterized.

Finally, the evaluation focused on single-hop and moderate complexity temporal queries. Performance on highly complex multi-hop temporal reasoning (e.g., “Who reported to the manager of Alice’s collaborator in 2020?”) requiring extensive graph traversal and temporal join operations remains an open question for future investigation.

5.4 Future Works

Future research can extend this work in the following directions:

1. **Cross-Domain Validation:** Evaluate TKG-RAG on diverse temporal domains including financial events (stock market changes, merger announcements), health-care knowledge (treatment guideline updates, drug approval timelines), political data (policy changes, governmental transitions), and scientific publications (break-through discoveries, research paradigm shifts). Cross-domain validation would confirm whether the observed 91.6% performance advantage generalizes beyond academic researcher profiling.
2. **Automated Entity Canonicalization:** Develop normalization layers incorporating fuzzy string matching, phonetic encoding (Soundex, Metaphone), and entity linking to external knowledge bases (Wikidata, DBpedia) to resolve surface-form variations. This enhancement could improve Exact Match performance beyond the
3. **Multi-Hop Temporal Reasoning:** Enhance graph traversal algorithms to support complex temporal joins and aggregations across disconnected subgraphs. Investigate neural-symbolic approaches combining graph neural networks with temporal logic for queries requiring extensive reasoning chains (e.g., “Track the evolution of collaboration patterns between research groups from 2015 to 2024”).
4. **Dynamic Knowledge Graph Updates:** Implement incremental update mechanisms supporting real-time fact insertion and invalidation without full graph reconstruction. Explore continuous learning paradigms where the system automatically detects temporal contradictions and propagates invalidations through dependent facts.

5.5 Concluding Remarks

This thesis demonstrates that temporal knowledge graphs fundamentally transform retrieval-augmented generation capabilities for time-sensitive question answering. Through systematic comparative evaluation, we have established that explicit temporal modeling is not merely beneficial but essential achieving 91.6% overall performance improvement and near-perfect answer accuracy (Exact Match: 0.797) while baseline systems without temporal awareness effectively fail on lexical precision metrics.

The research validates three critical design principles: (1) explicit temporal modeling through bitemporal validity tracking is essential for temporal QA, (2) structural graph relationships and temporal semantics are complementary rather than substitutable,

and (3) acceptable trade-offs in response verbosity are warranted when temporal accuracy improvements exceed 300%. These principles provide actionable guidance for researchers and practitioners developing knowledge-intensive AI systems in dynamic domains.

The dramatic performance differences observed particularly the 15,836% improvement on Exact Match over graph-based RAG underscore a fundamental limitation of current RAG architectures: treating time as unstructured metadata rather than first-class semantic constraints. As organizations increasingly rely on AI systems for decision-making in domains where facts evolve continuously financial markets, healthcare guidelines, regulatory compliance, scientific research temporal accuracy becomes not merely desirable but mandatory.

This work establishes both the feasibility and necessity of temporal knowledge graph integration in production RAG systems. The implemented prototype demonstrates that sophisticated temporal reasoning need not sacrifice system responsiveness or scalability, achieving sub-second query latency while maintaining comprehensive temporal validity tracking across thousands of entities and relationships.

While challenges remain particularly in cross-domain generalization, multi-hop temporal reasoning, and automated knowledge graph maintenance the overwhelming empirical evidence presented in this thesis confirms that temporal knowledge graphs represent a critical evolutionary step in retrieval-augmented generation. The path forward is clear: AI systems operating in dynamic information environments must incorporate explicit temporal semantics to achieve reliable, trustworthy, and accurate reasoning over evolving knowledge.

This thesis contributes not only a validated technical framework but also rigorous empirical evidence supporting the temporal knowledge graph paradigm. As Large Language Models continue their rapid advancement in language understanding and generation capabilities, the bottleneck increasingly shifts from linguistic fluency to factual accuracy and temporal correctness. By addressing this bottleneck through structured temporal knowledge representation, we move closer to AI systems that can reliably support human decision-making in complex, time-sensitive domains where accuracy and timeliness determine outcomes.

The research presented here concludes this specific investigation, but it simultaneously opens a broader research agenda toward temporally-aware AI systems. Future work spanning cross-domain validation, dynamic knowledge updates, complex temporal reasoning, and scalable deployment will build upon the foundational principles and empirical validation established in this thesis. The ultimate vision AI systems that reason accurately about evolving knowledge across arbitrary time scales and domains remains aspirational, but this work demonstrates that the vision is achievable through

principled integration of temporal knowledge graphs with retrieval-augmented generation.

Reference

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, *et al.*, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” *NeurIPS*, vol. 33, pp. 9459–9474, 2020.
- [2] S. Wang, Y. Zhu, H. Liu, Z. Zheng, C. Chen, and J. Li, “Knowledge editing for large language models: A survey,” *ACM Computing Surveys*, vol. 57, pp. 1 – 37, 2023.
- [3] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, Q. Guo, M. Wang, and H. Wang, “Retrieval-augmented generation for large language models: A survey,” *ArXiv*, vol. abs/2312.10997, 2023.
- [4] X. Zou, “A survey on application of knowledge graph,” *Journal of Physics: Conference Series*, vol. 1487, 2020.
- [5] S. Ji, S. Pan, E. Cambria, P. Marttinen, and P. S. Yu, “A survey on knowledge graphs: Representation, acquisition, and applications,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, pp. 494–514, 2020.
- [6] B. Cai, Y. Xiang, L. Gao, H. Zhang, Y. Li, and J. Li, “Temporal knowledge graph completion: A survey,” pp. 6545–6553, 2022.
- [7] X. Qian, Y. Zhang, Y. Zhao, B. Zhou, X. Sui, L. Zhang, and K. Song, “Timer4 : Time-aware retrieval-augmented large language models for temporal knowledge graph question answering,” pp. 6942–6952, 2024.
- [8] S. Ji, S. Pan, E. Cambria, P. Marttinen, and P. S. Yu, “A survey on knowledge graphs: Representation, acquisition, and applications,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 2, pp. 494–514, 2022.
- [9] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [10] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, and H. Wang, “Retrieval-augmented generation for large language models: A survey,” *ACM Computing Surveys*, 2024.

- [11] B. Cai, J. Huang, and X. Ren, “Temporal knowledge graph reasoning with time-aware neural models,” in *Proceedings of the 31st International Joint Conference on Artificial Intelligence (IJCAI)*, 2022.
- [12] C. Qian, Y. Wang, S. Li, and M. Zhang, “Time-aware question answering over temporal knowledge graphs,” in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.
- [13] I. Dasgupta, A. Lampinen, S. Chan, A. Creswell, and J. McClelland, “Language model agents: A survey,” *arXiv preprint arXiv:2308.11432*, 2023.
- [14] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “React: Synergizing reasoning and acting in language models,” *arXiv preprint arXiv:2210.03629*, 2023.
- [15] A. Bordes, N. Usunier, A. Garcia-Duran, J. Weston, and O. Yakhnenko, “Translating embeddings for modeling multi-relational data,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2013.
- [16] M. Nickel, K. Murphy, V. Tresp, and E. Gabrilovich, “A review of relational machine learning for knowledge graphs,” *Proceedings of the IEEE*, vol. 104, no. 1, pp. 11–33, 2016.
- [17] A. Hogan, E. Blomqvist, M. Cochez, C. d’Amato, G. Melo, C. Gutierrez, S. Kirrane, J. E. Labra Gayo, R. Navigli, S. Neumaier, A. Polleres, S. M. Rashid, A. Rula, S. Schlobach, J. Sequeda, S. Staab, and A. Zimmermann, “Knowledge graphs,” *ACM Computing Surveys*, vol. 54, no. 4, 2021.
- [18] J. Leblay and M. W. Chekol, “Deriving validity time in knowledge graph,” in *Proceedings of the 2018 World Wide Web Conference (WWW)*, 2018.
- [19] S. M. Kazemi, R. Goel, K. Jain, I. Kobyzev, A. Sethi, P. Forsyth, and P. Poupart, “Representation learning for dynamic graphs: A survey,” *Journal of Machine Learning Research*, vol. 21, no. 70, pp. 1–73, 2020.
- [20] W. Jin, T. Ma, P. Cui, and B. Wang, “Temporal knowledge graph completion: A review,” *arXiv preprint arXiv:2004.07202*, 2020.
- [21] R. Trivedi, M. Farajtabar, P. Biswal, and H. Zha, “Know-evolve: Deep temporal reasoning for dynamic knowledge graphs,” in *ICML*, 2017.
- [22] G. Izacard and E. Grave, “Leveraging passage retrieval with generative models for open-domain question answering,” *arXiv preprint arXiv:2007.01282*, 2021.

- [23] S. Borgeaud, A. Mensch, J. Hoffmann, and et al., “Improving language models by retrieving from external knowledge,” *arXiv preprint arXiv:2204.05862*, 2022.
- [24] Z. Huang, S. Yao, and et al., “Tree of thoughts: Deliberate problem solving with large language models,” *arXiv preprint arXiv:2212.08073*, 2022.
- [25] S. Yao, Z. Huang, and et al., “Agent-based reasoning in large language models: Frameworks and applications,” *arXiv preprint arXiv:2303.08894*, 2023.
- [26] P. Zhao, H. Zhang, Q. Yu, Z. Wang, Y. Geng, F. Fu, L. Yang, W. Zhang, and B. Cui, “Retrieval-augmented generation for ai-generated content: A survey,” *ArXiv*, vol. abs/2402.19473, 2024.
- [27] H. Yu, A. Gan, K. Zhang, S. Tong, Q. Liu, and Z. Liu, “Evaluation of retrieval-augmented generation: A survey,” *ArXiv*, vol. abs/2405.07437, 2024.
- [28] A. J. Oche, A. G. Folashade, T. Ghosal, and A. Biswas, “A systematic review of key retrieval-augmented generation (rag) systems: Progress, gaps, and future directions,” 2025.
- [29] X. Qian, Y. Zhang, Y. Zhao, B. Zhou, X. Sui, L. Zhang, and K. Song, “Timer⁴: Time-aware retrieval-augmented large language models for temporal knowledge graph question answering,” *EMNLP*, pp. 6942–6952, 2024.
- [30] A. N. P. Sun and Others, “Dyg-rag: Dynamic graph retrieval-augmented generation for temporal question answering,” *arXiv*, vol. abs/XXXX.XXXXX, 2025. Under review; replace with official citation when available.
- [31] A. N. P. Zhang and Others, “E²-rag: Entity-event retrieval-augmented generation for temporal and causal reasoning,” *arXiv*, vol. abs/XXXX.XXXXX, 2025. ChronoQA-based; update venue when published.
- [32] A. N. P. Ma and Others, “Think-on-graph 2.0: Iterative knowledge graph and text retrieval for deep multi-hop reasoning,” *arXiv*, vol. abs/XXXX.XXXXX, 2024. Update when final publication is available.

Appendices (Optional)

Include in the appendices information that could not be included in the formal body of the report because it would disrupt the continuity of the discussion. Background materials, experimental data tables, and extra documentation should be placed in the appendix.

Please use the following page format for the hard-cover binding of your report.

TKG-RAG: Knowledge Graph Incorporated with Time Domain for Institutional Setup

by

Hasimunnahar Shanta - 21002585

Jahidul Islam -221002504

Rifat Hossain Abrar - 21002385

Thesis for the degree of

Bachelor of Science in Computer Science and Engineering



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

GREEN UNIVERSITY OF BANGLADESH

FALL 2025